Multimedia Tools and Applications (2024) 83:51469–51512
https://doi.org/10.1007/s11042-023-17308-3

# Digital forensic framework for smart contract vulnerabilities using ensemble models

**Lohith J J[1] · Kunwar Singh[1] · Bharatesh Chakravarthi[2]**

Received: 6 March 2023 / Revised: 21 September 2023 / Accepted: 25 September 2023 /
Published online: 13 November 2023
© The Author(s), under exclusive licence to Springer Science+Business Media, LLC, part of Springer Nature 2023

**Abstract**
Forensic identification of vulnerabilities in Ethereum based smart contract has received significant interest among security agencies and researchers because it normally manages a lot of cryptocurrencies worth billions of dollars. Recalling the fact that smart contracts, a small set of instruction or programmes especially designed towards user verification, authentication and access control in Blockchain, can have the different vulnerabilities including Denial of Services (DoS), access control, Arithmetic integer overflow, arithmetic bad-randomness, re-entrancy, unchecked low-level calls, etc. In this paper a novel methodology using natural language processing and machine learning based vulnerability detection system is developed. The overall proposed model was designed towards Ethereum based Smart Contracts, where the standard benchmark data with the different vulnerability types has been taken into consideration. Though, classical methods apply SolMatrix tool to extract the features from Ethereum Solidarity vulnerability matrix however, they are confined to address feature variations, data imbalance which eventually impacts overall (vulnerability) classification performance. Considering this fact, in this research the overall methodology was defined in such manner that it assesses a Ethereum Solidity Smart Contract towards the probability of any (aforesaid) vulnerability types.To examine the performance of the proposed vulnerability detection model, we obtained results in terms of Accuracy, F-Measure, and Area and ROC Curve (AUC). The extensive performance analysis in terms of intra-model comparison, the average accuracy with SMOTE sampled data resulted near 90% of accuracy using Random Forest Algorithm. AUC performance too was found near 0.7, confirming acceptability of the proposed model. With Random Forest algorithm the average F-Measure was found to be near 0.86.

✉ Bharatesh Chakravarthi
chakravarthi.bsb@asu.edu

Lohith J J
lohithjj@gmail.com

Kunwar Singh
kunwar@nitt.edu

[1] Dept. of Computer Science and Engineering, National Institute of Technology, Tanjore Main Road, 620015 Tiruchirapalli, Tamilnadu, India

[2] School of Computing and Augmented Reality, Ira A. Fulton Schools of Engineering, Arizona State University, 85281 Tempe, Arizona, United States

**Keywords** Blockchain · Ethereum · Machine learning · Natural language processing · Smart contract · Vulnerability

## 1 Introduction

Blockchain technology (BC) in recent years, has revolutionize a variety of industries such as healthcare [1, 2], financial [3, 4], insurance [5], government [6], real estate [7], social media [8, 9], supply chain [10, 11], and education [12], etc. It efficiently and logically answers a number of interesting real-world challenges. The term BC denotes a significant shift in how data is acquired, stored, and delivered due to 3 key characteristics: P2P network, decentralization,and immutability. Whereas the 1st generation of BC technology is almost purely used for cryptocurrencies like Bitcoin. Bitcoin according to author Andreas Antonopoulos, un-banks the banked or banks the un-banked [13, 14], the 2nd generation, demonstrated by Ethereum, is an open and decentralized platform that enables a new computing paradigm [15, 16]. These complex system's semantic eventually introduce dozens of new security flaws that don't exist in pure monetary systems like Bitcoin. Ethereum is a widely preferred trade platform that allows an account holder to perform exchange, buy, and sell cryptocurrency with the help of smart contracts, these contracts automatically executed when preset terms and conditions are satisfied and are maintained in BC [17, 18]. Contracts have the unique ability to transfer ether to and from users as well as other contracts. Users submit transactions to the Ethereum network for creation of new contracts, perform contract functionalities, and send ether to contracts or other participants. All of the transactions were indeed recorded on the blockchain, which is a public, allows to append the state of transaction on a data structure. The order in which transactions are recorded on the blockchain establishes the state of every contract also each user's balance. The smart contract's security has received significant interest among security agencies and researchers because it normally manages a lot of cryptocurrency worth billions of dollars.

Practically, BC platforms enable business owners in developing apps that use BC concepts. Because not everyone or every firm has the resources (Qualified and experience developers, time, money, and infrastructure) to build their own BC from the initial concept, these businesses rely on BC platforms established by IT behemoths for quicker and efficient implementation of their software application. Every month, a new BC platform with additional capabilities and the similar backend concept is introduced. We illustrates the popular BC platforms with important parameters like environment in which smart contracts can run, programming language used to develop the platform, conscious mechanism, protocol which guarantee that online transactions are viable, prevent the potential of fraud or waste, and ensure that participants are not going to cheat each other. BC and smart contract development environments are still relatively new fields, there is a lack of certainty and conflict about them. Hence many businesses selects a smart contract platform based on its experience and reputation of developer community (Fig. 1). As Ethereum is first platform, it has gained popularity since its launch,and has been able to attract investment and support from multi national companies such as Intel and Samsung. To understand the importance of Ethereum, we surveyed the various websites and found current number of Ethereum Transaction records Per Day is 1.175 million as shown in Fig. 2 [1], While Ethereum has several benefits and advantages over its less well-known competitors. The daily number of transactions done on the Ethereum network is represented as Ethereum Transactions Per Day. Even as market for the

[1] https://ycharts.com/

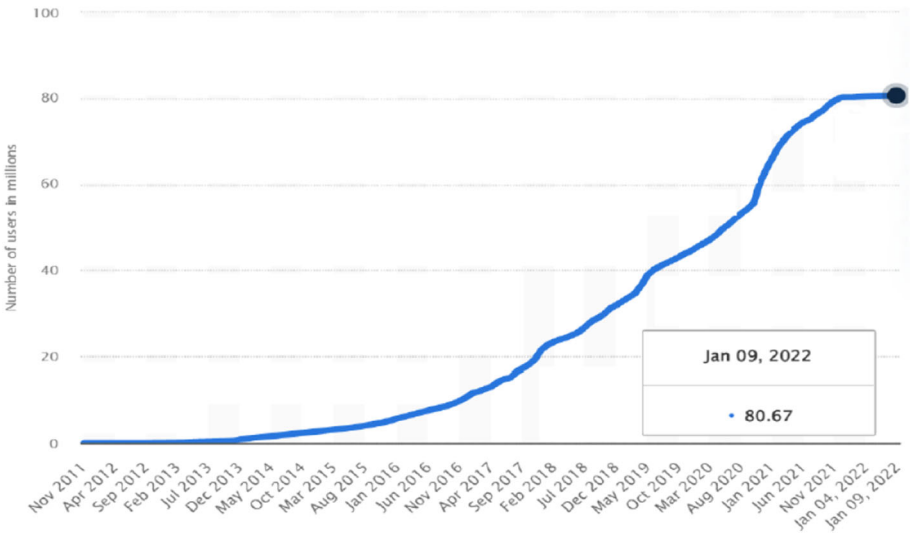



**Fig. 1** Number of blockchain users

underlying token rises, higher interest or trust in Ethereum potentially lead to more transactions which is evident from the above Fig. 2. The Ethereum Virtual Machine (EVM) is the programme that determines how to compute a new valid state from block to block. The EVM is a robust, sandboxed virtual stack that executes contract bytecode and is incorporated within entire Ethereum node .

## 1.1 Smart contracts terminologies

The smart or self-executing contract is a computer software (cryptographically signed instructions) that can assist Ethereum in encoding rules or scripts for processing transactions. These contracts act as a basic protocols for verifying and creating new "blocks" of content on the BC. Nick Szabo introduced the term "smart contracts" in the early 1990s, referring to a set of contracts, stated in digital form, containing rules wherein the parties fulfil on each agreements [19]. Satoshi Nakamoto and Martti Malmi[2] created the world's 1st Electronic Cash System or digital currency, known as Bitcoin, in the year 2008 [20], without adopting the concept of single administrator or central bank control, by implementing the 1st BC application. In order to provide a distributed ledger that stores transactions, Bitcoin relies on cryptographic "proof of work" procedures, digital signatures, and peer-to-peer distributed networking layer.

### 1.1.1 Advantages of using smart contracts are

**Processing time is short and accuracy is more** Whenever a criteria is fulfilled, the contract is instantly executed. There is really no manual documentation or invoice pertaining to entire work, hence time will not be wasted in fixing errors that can occur due to manual process, since smart contracts are digitized ,autonomous and precise. All the account holders involved

[2] https://bitcoin.org/en/about-us#own

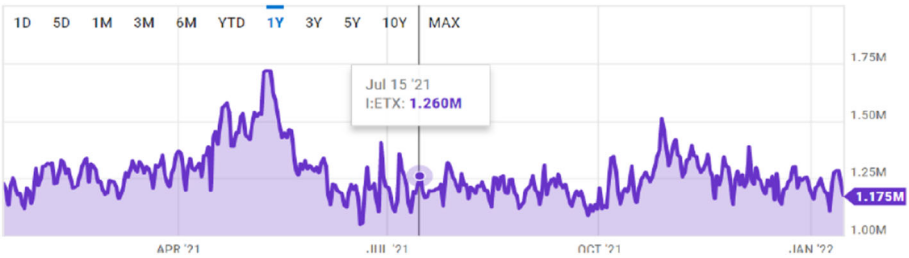



**Fig. 2** Number of transaction on Ethereum platform

in a transaction can see the result instantly without any delay or third party's involvement. It is also possible to automate the workflow, targeting the next action when condition is satisfied.

**Trust worthiness** Unnecessary taking tension about privacy and security of personal data has no room. There is no chance of any manipulation to the participant's data as it is kept confidential and all transactions, events and records are encrypted end to end, no involvement of any middleman or third party agency. Only cryptographic content is shared among account holders involved in a transaction.

**Tamper-proof** It is not simple and easy to hack any data, because blockchain is built on such a concept that distributed ledger entries are broken into several blocks and each block is connected with its predecessor and successor block. Even if some one tries to hack, they must alter the entire chain, in order to modify single data record.

**Literally permanent** Because the system is considered to be highly redundant and exists on several network hops or node or participant systems, the failure of one of these has no impact on the system's functionalities.

**Indefensible** Refers to the assurance of every block of data is explicitly created by someone & they cannot deny the same (Table 1).

### 1.1.2 Popular smart contract platforms

## 1.2 Security flaws

No doubt that Smart contracts on Ethereum network are popular but there are a few security concerns. Several flaws were uncovered by researchers, putting thousands of dollars at stake. This is alarming news, but there is a bright aspect to it as well. That's the reason still Ethereum is ruling the market.

Smart contracts cover a wide range of topics. Although templates can be utilized, any extra code poses a security risk. Since there are lot of Ethereum-based contracts out there, it's important to determine if they're safe. Figure 3 shows how an attacker tries to attack smart contracts. Researchers hope to raise security standards by using a new approach to sniff out flaws. Unfortunately, already uncovered over 3,000 contracts that are currently susceptible. At current Ether pricing, these are worth approximately $6 million. Things might swiftly

**Table 1** Comparison of 8 best platforms for Smart Contract

| Platform | Environment to Run | Smart Contract Language | Permission Type | Protocol | TPS | Crypto currency |
|---|---|---|---|---|---|---|
| Ethereum | EVM | Solidity | Public | PoW | 14-27 | Ether |
| Solana | LLVM | Rust, C | Consortium | PoH | 65000 | SOL |
| EOS | WebAssembly | C++ | Public | BF,DPOS | 10-15 | EOS Token |
| Tezos | Tesoz VM | Micheison | Public | PoS | 40 | tez,XTZ |
| Polkadot | PEE | Depends on Selected chain | Depends | NPoS | 1000 | DOT |
| Stellar | Docker | C++, Scala Not,Go | Consortium | FBA | 1000 | Lumen |
| NEO | NEOVM | C# | Public | PoS dBFT | 10000 | ERC20 |
| Hyper-ledger | Docker | Go, Java Script | Provate | CFT | 3500 | SDK |

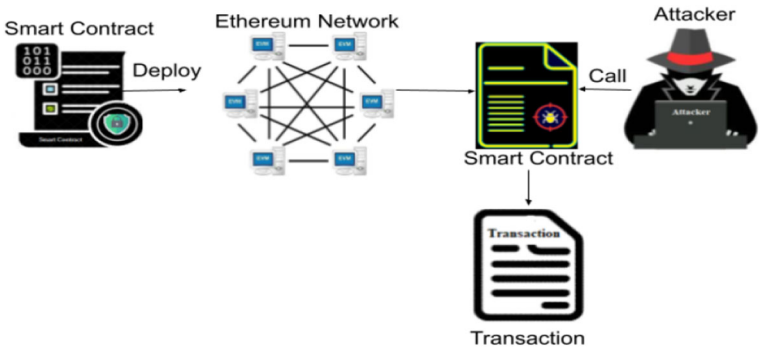



**Fig. 3** How attacker uses Ethereum network

spiral out of control if someone took advantage of these weaknesses. The key issue is how smart contracts are utilized to manage the currency of other individuals. Even though it may appear to be interesting, there is always a cost. In reality existing contracts cannot be altered, this is a serious problem. There is little that can be done, especially if a security vulnerability is uncovered. For whatever reason, this is a severe design problem.

### 1.3 Motivation

Existing literature has resulted in the formation of many taxonomies related to Smart Contract vulnerabilities. The conventional strategy is to categorize vulnerabilities with respect to Ethereum Virtual Machine(EVM), BlockChain, and Solidity-related concerns. The first three are larger classes that provide readers with little insight regarding the actual code's major weaknesses. In the same way, the survey in [21] categories smart contract vulnerabilities in terms of EVM and Solidity. Another important survey grouped the vulnerabilities by layering such as application, data, and consensus layers [22]. Furthermore, the study in [23] classified bugs using the NIST Bugs Framework. Smart contract, on the other hand, brought new types of vulnerabilities [24] that were rare in conventional programming model.

### 1.4 Our contribution

We are the first to classify smart contract's security problems based on the domain knowledge of the vulnerable operation. A thorough knowledge of vulnerabilities necessitates the use of coding examples, however we focus on a top vulnerability collection from DASP. It's worth noting that the most recent survey on smart contract attacks was released in 2017. The Solidity programming language has evolved significantly since then. We use the ".sol" Solidity format files and convert them into csv format for further processing. Most of the classical methods apply SolMatrix tool to extract the features from Ethereum Solidarity vulnerability matrix however, are confined to address feature variations, data imbalance which eventually impacts overall (vulnerability) classification performance. Considering this fact, in this research the overall methodology was defined in such manner that it assesses a Ethereum Solidity Smart Contract towards the probability of any (aforesaid) vulnerability types. We have designed a hybrid system with blend of two popular Artificial intelligence based technologies such as

natural language processing for feature engineering and machine learning for classification modeling.

### 1.5 Organization of paper

The rest of the paper is organized as follows. Section 2 briefs the machine learning algorithms used and methodology implemented in the paper. Section 3 discusses about Results, Finally Section 4 concludes. On the basis of essential factors, a scientific and detailed comparison is also made in Table 6. These critical factors have an influence on the system's quality and acceptability.

### 1.6 Related work

**Oyente** was developed by Loi Luu and his team from National University of Singapore in the year 2016 [27]. It is the first smart contract security tool. This tool retrieves the control flow graph from a contract's EVM bytecode and symbolically executes it to find vulnerability patterns. It checks blocks of code that are likely to be vulnerable due to their structure, calls to external contracts, or poor management of execution exceptions by symbolically executing the EVM bytecode of smart contracts. Oyente can detect the Parity Multi-signature Bug, among other concerns, like Re-entrancy issue, Callstack Depth Attack, and Timestamp Dependency. Some recent papers suggest approaches for detecting vulnerabilities in contract program using static analysis [28].

**Solograph** Produces a DOT graph that depicts the function control flow of a Solidity contract while highlighting potential security flaws [29]. To aid users in their analysis, Solgraph visualises the call flow in Solidity contracts [30]. It accepts Solidity code and generates a network with nodes corresponding to operations and directed edges corresponding to function calls. Colors across nodes indicate attributes which send to outside external address or payable [31]. This command-line interface tool is built in JavaScript and has been published in Github since July 2016 under an ISC licence. The AST is generated using the Solidity compiler, and the graph is displayed using GraphViz. It illustrates various actions, each of which is represented by a distinct color as follows: Send to an external address in red. Blue indicates that the function is constant. View is indicated by the color yellow. Green is the color of purity. Call in the colors of orange. Purple is the color of transfer. Last Lilac for Payable.

**Mythril** is a Python command-line framework for interactively examining smart contracts. It visualises the Control flow graph (CFG) and processes EVM bytecode visually, with the nodes holding decomposed code as well as the edges indicated by path formulas [32]. To trim the search space & calculates concrete values for exposing particular vulnerability, by satisfiability modulo theories (SMT) solver Z3. ConsenSys developed and maintains Mythril, which has been available on Github since September 2017 under an MIT licence.

EVM bytecode is a low-level computer language created by compiling a high-level programming language like Solidity. EVM is a virtual machine that sits between the operating system and the application layer to reduce OS dependency. Ethereum smart contracts, can run nearly on any computer because of bytecode. Static analysis is a set of ways for examining a contract's source code or bytecode without running it. The majority of the methods listed

**Table 2** Top 10 open source security tools for smart contracts

| Tools | Solidity | Byte Code | Dynamic Analysis | Static Analysis | Formal Verification |
|---|---|---|---|---|---|
| Oyenete | No | Yes | No | No | Yes |
| Solgraph | Yes | No | No | No | No |
| SmartCheck | Yes | No | No | No | No |
| Mythril | No | Yes | No | Yes | Yes |
| ConractLarva | Yes | No | Yes | No | No |
| EtherIR | No | Yes | No | No | yes |
| EtherTrust | No | Yes | No | No | yes |
| Slither | Yes | Yes | No | Yes | No |
| Osiris | No | Yes | No | No | Yes |
| Vandal | No | Yes | No | No | Yes |

here are static. Dynamic analysis refers to observing a contract while it is being executed (in part) in its original context (Table 2).

### 1.7 Analysis based on NLP techniques

To find vulnerabilities, the previous approach relied on classical program analysis techniques and fuzzing. In fact, in [33], author used string-based vulnerability signatures. These were generated utilizing an automated string analysis framework. Another research [34] used a statistical language model aim to minimize the task to an NLP problem. Then, to interpret source code in diverse jobs, multiple NLP approaches were developed and applied. Experts viewed code as text and utilised natural language processing (NLP) to infer the presence of vulnerabilities. The inference of algorithms was used to expose compiler issues in [35]. Techniques of neural machine translation have been used in [36]. In an Encoder-Decoder paradigm, the extraction of bug fixes pattern was presented to infer the presence of flawed code and repair it. TP-Detect [81] is one novel Machine Learning based approach, TriPix dataset which is a combination of Opcode Trigram values and Pixel Values extracted from the opcodes (Table 3).

### 1.8 Smart contract vulnerability

Smart contracts code is rather sophisticated, despite the fact that they are supposed to ease operations. Given the fact that every contract is authored by a human coder, their input is impossible to decipher. As a result, severe security issues might take months, if not decades, to disclose. To uncover weaknesses, the researchers take a different technique. In recent years, several methodologies for analyzing smart contract vulnerabilities have been proposed [37]. Current methods all rely on static analysis and symbolic execution to assure the safety and validity of smart contracts. These are complex, take a long time to analyze, lack pertinence, have a high amount of false positives, are difficult to scale to large contracts, and do not cover all types of well-known vulnerabilities [38].

**Table 3** NLP-based analysis

| Year | Feature | Vulnerability Type | Algorithm | Accuracy | Ref |
|---|---|---|---|---|---|
| 2021 | Contract & Total Participants Balance | Re-entrancy | Single-Function, Cross-Function Attack | NA | [71] |
| 2021 | Clone Detection | Solidity | Eth2Vec,SVM | 58.20% | [72] |
| 2021 | BERT | Solidity | SVM, Eth2Vec, DR-GCN | 90.90% | [73] |
| 2020 | Feature Space | Re-entrancy, Exception Handling, Transaction Ordering, Integer Overflow Underflow Timestamp Dependency | N-gram, SMOTE | 96% | [74] |
| 2018 | LLVM | Re-entrancy,Exception Handling, Transaction Ordering, Integer Overflow/Underflow, Timestamp Depency | RAFT/Paxos | 94.60% | [75] |
| 2017 | Time Constrain | Smart Contract Toxonomy | Transaction by Category | 94% | [76] |

Top 10 type of vulnerabilities[3,4]

1. Reentrancy
2. Access Control
3. Arithmetic Issues
4. Unchecked Return Values For Low Level Calls
5. Denial of Service
6. Bad Randomness
7. Front-Running
8. Time manipulation
9. Short Address Attack
10. Unknown Unknowns

Since the launch of bitcoin almost 12 and half years ago, cryptocurrency as well as other BC virtual assets have always been the primary focus of the most of attacks including a variety of threat vectors. As smart contracts are adopted for FinTech as well as enterprise BC applications, several of these potential attacks will target identical weaknesses.

***Reentrancy*** A process is said to be re-entrant if it can be paused in the mid of its execution, restarted (re-entered), but both executions terminate without errors. Reentrancy within that framework of Ethereum smart contracts can lead to major flaws. One of the most devastating attacks in any Solidity smart contract is indeed the Reentrancy attack. Whenever a function initiates an external call to some other untrusted contract, it is called a reentrancy attack. Then, in an attempt to drain funds, the untrustworthy contract does a recursive call back to the initial function.

**Listing 1** Malicious attacker's contract

```
DFunds public depositFunds;
constructor(address _depositFundsAddress)
{
depositFunds =
DFunds(_depositFundsAddress);
}
// Fallback is called when DFunds
//sends Ether to this contract.
fallback() external payable {
if(address(depositFunds).balance >= 1 ether)
{
depositFunds.withdraw();
}}
function attack() external payable {
require(msg.value >= 1 ether);
depositFunds.deposit{value: 1 ether}();
depositFunds.withdraw();
}}
```

Ethereum smart contracts can exchange Ether towards each other. When a transaction receives a request containing Ether but with no data and none of the function is specified by the initiator, then fallback function (a default unnamed function), is called. Control is

[3] https://dasp.co/

[4] https://pentestwiki.org/academy/top-10-smart-contract-vulnerabilities-on-ethereum/

transferred to contract B when payments are transferred from contract A to contract B [39]. During the timeframe B has control, this can call for any public function of contract A, including the one that called B. Reentrancy is the term used for this condition [40].

External contracts have the potential to seize control of the sequential flow, which is one of the key risks of using them. A reentrancy attack (also known as a recursive call attack) occurs when a malicious contract calls back into the calling contract before the function's initial invocation is completed as explained in prior paragraph. This may lead the function's different invocations to interact in unfavourable ways. The software allows for a session in which an actor must do many behaviours, but it does not verify that the behaviours are completed in the correct order. The following are examples of workflow-related behaviours: The steps are must be carried out in the correct order.

Steps that are necessary should not bypassed.

Steps should not be disturbed in between.

Steps must be completed in a timely manner.

***Impact of this attack*** Based on whether they've placed Ethereum-based cryptocurrencies or tokens on an internet platform that stores money, such types of vulnerabilities can affect a lot of people. Crypto gambling sites, loan services, Ethereum banks, and other online services where customers are likely to store their coins or tokens fall under this category. Users could lose money if an attacker designs a bad smart contract and assaults those sites.

***Access control*** As the smart contract is developed, the owner must appropriately configured, the sufficient controls to allow specified functions for execution.The functionality of a contract is normally accessed over its public or external functions [41]. Access control bypasses are more subtle than insecure visibility settings, which offer attackers easy access to a contract's secret data or logic (Table 4).

***Integer underflow and overflow*** The word length used in Ethereum based smart contracts is 256 bits. In practice, this means that the permitted numbers on Ethereum contracts range between 0 to 4,294,967,295 (or $2^{256}$), or nearly 4.3 billion, with no negative values permitted.They are particularly problematic in smart contracts, since unsigned integers are common and also most developers are just used to work with normal int types. Many seemingly benign code become channels for theft or loss of service if overflows occur [42]. If 1 is added to the top of the range in an integer overflow, it will cycle back down to zero ($2^{256} + 1 = 0$).

**Table 4** Recent attacks

| Hack | Year | Lost in (Million$) | Cause of Attack |
|---|---|---|---|
| Uniswap/Lendf | Apr 2020 | 25 | Reentrancy attack |
| BurgerSwap hack | May 2021 | 7.2 | bogus_token contract. |
| SURGEBNB | Aug 2021 | 4 | Price manipulation. |
| Cream Finance | Aug 2021 | 18.8 | Got second loan |
| Siren protocol | Sept 2021 | 3.5 | Reentrancy attack |
| Grim Finance Defi | Dec2021 | 30 | Reentrancy attack |
| dForce | Jan2022 | 25 | Reentrancy attack |

According to an integer underflow, the opposite is true: subtracting 1 from zero causes the range to cycle all the way to the top ($0–1= 2^{256}$).

***Unchecked CALL*** The fundamental notion behind this vulnerability is that the message call's return value isn't examined. As a result, and even when the called contract throws an exception, execution will continue. If the call fails by accident or because an attacker induces it to fail, the computer logic that follows may behave unexpectedly. Call, call code, delegate call, and transmit are few low-level call functions in Solidity, that work with raw addresses. These low-level functions never throw an exception, but if the call encounters one, they will return false [43]. External calls can be made in a variety of ways in Solidity. The transfer function is generally used to send Ether to external accounts. Alternatively, the send function can also be utilised, as well as the CALL opcode in Solidity could be used directly for more powerful and flexible external calls. The call and transmit functions both return a Boolean value indicating whether or not the call was successful [44]. As a result, these functions include a simple caveat whereas if external call (initialised by call / send) fails, the transaction that performs these procedures will not revert instead, the functions would also simply return false. A typical mistake is not checking the return value because the programmer expects a reversion unless the external call fails.

***Denial of service (DoS)*** Gas price plays an important role in smart contract. Denial of service is extremely dangerous in the Ethereum network while other kinds of attacks can recover quickly, smart contracts can be knocked offline permanently by one of these attacks. DOS can occur in a variety of ways, such as maliciously behaving as the recipient of a transaction, simply raising the gas needed to calculate a function, misusing access controls to gain access to private elements of smart contracts, exploiting mixups and breach, and so on [45]. In this type of attack the required price of gas becomes higher than the maximum allowed price. As a result, the deal cannot be completed.

***Bad randomness*** Randomness is required in several financial operations for example, to construct a cryptography, gaming, gambling, lottery, a competitive sports, or crypto-collectibles. Unfortunately, creating the random numbers on a public blockchain is difficult [46]. Computations must be predictable in order to be replayed in decentralized method, and any data that can be used as a source of randomness is also accessible to an attacker or malicious users, they can replicate it and attack the method relying on its unpredictability.

***Front-Running*** In a transaction any participant can fix or set greater fees than others. By sending their transaction with larger costs to preempt the original transaction, a rogue user can overrun another transaction [47].

***Timestamp dependence*** Because a smart contract runs on the EVM, the insight it gives is limited to the smart contract's activities and blocks. Usually it will not provides any detail information about its environment, such as the host OS, IP address, or even the time. A smart contract programmer would look for the details inside the block's metadata's timestamp feature. Moreover, the timestamp feature in the block is random, as well as the block's miner can put any date they wants without further verification from all the other network nodes [48]. A bad miner could fool a smart contract that depends on such timestamp data. A shorthand reference to the present block's timestamp in the Solidity language, which is the measure of time since the Unix Epoch when the function's transaction was posted to the blockchain. As a result, the time is solely determined by the clock on the computer mining that specific block. A miner is supposed to provide a precise clock in general. Users will have problems

connecting to peers if their clock is more than 12 seconds out of sync with other miners' clocks [49].

**Listing 2** Timestamp Manipulation Vulnerability

```
external payable returns (bool) {
if (yourGuess == now % 2 > 0) {
uint fee = msg.value / 10;
msg.sender.transfer(msg.value * 2 - fee);}
}
```

***Short address attack*** Short address attacks are a result of the EVM's acceptance of improperly filled arguments. Because smart contract addresses typically uint256, if a developer just uses uint, then EVM might fill the remaining bytes with zeroes, causing the recipient address to be manipulated [50]. Attackers can take advantage of this by employing uniquely engineered addresses to trick clients into wrongly encoding arguments prior to using them in transactions.

***Unknown unknowns*** Solidity is a brand-new programming language, thus anything could happen. A few of the most dangerous smart contract flaws caught everyone off guard, and there's no reason to think there won't be another one that's just as surprising or as catastrophic [51]. Post-conditions on tokens, allowing users to protect their funds against unknown attackers and unknown faults in other contracts.

For the proposed model we have considered the logical vulnerabilities from smart contracts as shown in Fig. 4.

### 1.9 Challenges

1. 51 % Attack Over Bitcoin Network The 51 percent attack on a BC network: is a method of forking a BC in order to double spend [52]. This is the biggest security concern of blockchain [53], which are hypothetical incidents in which attacker can undo the transactions in alternate block of a side chain or branch of such BC or hide information which is occurring inside the main chain [54]. The chance of miners who are mining

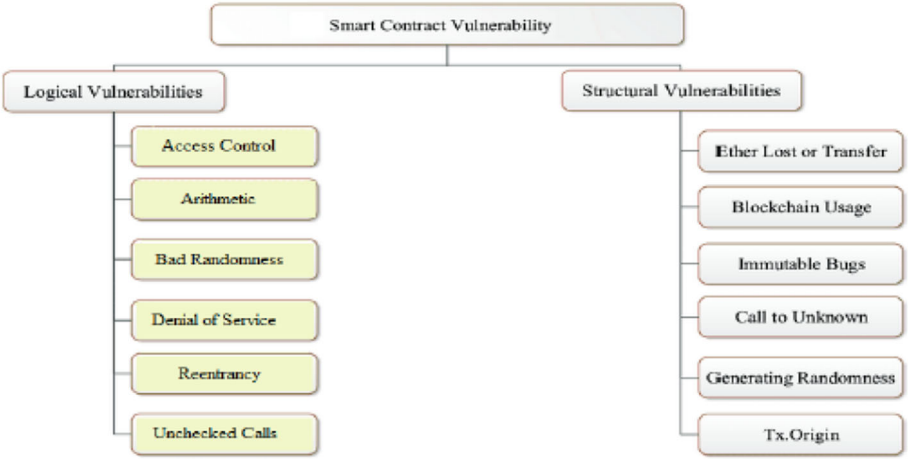



**Fig. 4** Taxonomy of Vulnerabilities in Smart Contracts

blocks is determined by a PoW. Miners work with each other to mine larger number of blocks and provide more computational power to maintain network control. Any account holder participating in a transaction with more than 51 percent processing power can discover random or pseudo-random numbers also known as the number only used once (nonce) [55], which will aid miner(s) in determining which blocks are allowed and which one are not allowed [56]. It will assist the attacker in altering transactions which might result in a double-spending attack, as well as preventing the blocks being verified by the transactions [57]. However, gaining 51 percent hashing power is not always needed to compromise the blockchain network, a double-spending attack can be carried out with lesser than half of the hashing power, but the chances of success are very less [58, 59].

2. Based on Decentralization: Although the decentralised nature of networks enables peer-to-peer activities[5], also it creates serious privacy concerns for personal data[6]. Author[60] examined a few security and privacy challenges in the banking industry, including threats, malevolent adversaries, and cyberattacks. Distributed personal information management was investigated by author [61] Studied the data privacy issues for personal data.
3. Uncertain Terminology: Often the keywords Distributed Ledger Technology "DTL" & "BC" are ambiguous [62]. In simple words, there seems to be lack of technical knowledge and experience among prospective customers, business owners, and government agencies [63], especially in areas like

   - Employment opportunities in BC,
   - Job market in DTL,
   - BC - based smart contracts that needs; business logic aspect of ledger accounts to be taken automatically,
   - Recognizing how and where to search for the required talent, and
   - Making investments in blockchain careers irrespective of demand for new skill.

4. Instead of attacking the entire network, an eclipse attack targets a decentralized network that can separate the specific participant(s) [64].
5. A human being's action is not error-free. Human coding flaws may serve as gateways for various attacks and threats in any blockchain application [65].
6. Whenever the agreement or contract receives lesser data than intended, then the EVM puts zeros at the ending of the address to guarantee the 256-bit data types, this is known as a short Address Attack [66].
7. The speed of blockchain-based transactions is extremely slow, which is a huge worry for businesses [67].
8. Due to the complex nature of economic and political regulations, implementing BC applications in various sectors is extremely difficult, since no central bank policy exists at present [68].
9. Attacks depend on the presence of a vulnerabilities and are implemented in software via web services, cryptographic protocols or smart contract, or in BC technology, identifying the vulnerability in smart contract is very challenging [69], these weaknesses allows the attacker to further exploit the application. These flaws can be any number of vulnerabilities such as Denial of Services (DoS), time-manipulation (or timestamp), access

[5] https://lisk.com/what-is-blockchain

[6] https://www.rt.com/usa/221919-obama-privacy-student-consumer/

control, arithmetic bad-randomness, front running, re-entrancy, short addres, unchecked low-level calls, etc.

## 2 Methodology

The methodology comprises of two stages, first one uses natural language processing (NLP) techniques. Second ensemble modeling for vulnerability identification. The algorithm for the detailed methodology is given below in Algorithm 1. More details are discussed in following sections.

Any numerical (Discrete/Continuous/ Interval/ Ratio) data will be processed easily by computer system, than categorical or textual (Nominal /Ordinal) data. Categorical data is often used to categorized information with comparable qualities, whereas numerical data represents information as numbers. It depicts key information using quantitative values, whereas textual data expresses information using a descriptive method.

The reason of choosing NLP for stage-1 is that smart contract dataset consists of small programs which are almost like English language statements along with mathematical operators and special characters. We are aware that machines can interpret language-based massive volume of unstructured data using NLP just like humans can, without becoming bored and in a consistent and unbiased manner. The aim is to use a representation of textual words, that conveys semantic and syntatic links related to logic or language. The numeric representation of words is called as embedding.

In order to detect the vulnerability from dataset with high accuracy, we decided to use pre-trained word embedding from Goolge News and Glove. Pre-trained Words are embeddings which have already been learned in either a previous trail or some identical situation and can be used to tackle a similar problem. Because of this reason, they are considered as Transfer Learning. Such embeddings are made by training it on large-scale dataset, preserving, and then applying them to new challenges.

We have selected Python programming language to carryout research, as it provides a large choice of tools and frameworks. Many of them can be found in the Natural Language Toolkit-NLTK or Gensim, which are open source collection of libraries, tools, and educational resources for developing NLP programs.

### 2.1 Stage-1

In our proposed model as shown in Fig. 5 of stage-1 which uses NLP is further divided into two phases: data cleaning and algorithm development for feature extraction. NLP Alone is not sufficient to detect vulnerability. NLP Primarily used because vulnerabilities in smart contracts are related to code logic and not natural language descriptions. So it is used in stage 1 to convert the smart contract code into vectors so that we can use it in Machine Learning Models. Language is the most compelling embodiment of intellect, and artificial-intelligence is very fascinating topics of research today. Hence we have adopted the same to detect the presence or absence of vulnerability from smart contracts. NLP has become increasingly vital and popular choice among researchers in making sense of text data and is a popular choice for building software such as, Topic discovery and modeling, Contextual extraction, Information retrieval, Text classification. However, unlike classical approaches

**Algorithm 1** Vulnerability Detection Algorithm

```
procedure PREPROCESS_DATA(input_contracts)
    vulnerability_categories = 'Access Control': 0, 'Arithmetic': 1, 'Bad Randomness': 2, 'Denial of Service': 3, 'Reentrancy': 4, 'Unchecked Low-Level Calls': 5
    preprocessed_data = []
    for contract in input_contracts do
        vulnerability_category = analyze_contract(contract)
        preprocessed_data.append('source_code': contract, 'vulnerability_category': vulnerability_category)
    end for
    save_as_csv(preprocessed_data, 'preprocessed_data.csv')
end procedure
procedure EXTRACT_FEATURES(input_csv)
    preprocessed_data = load_from_csv(input_csv)
    tfidf_features = calculate_tfidf_features(preprocessed_data)
    word2vec_features = calculate_word2vec_features(preprocessed_data)
    save_as_csv(tfidf_features, 'tfidf_features.csv')
    save_as_csv(word2vec_features, 'word2vec_features.csv')
end procedure
procedure DATA_SAMPLING(tfidf_csv, word2vec_csv)
    tfidf_features = load_from_csv(tfidf_csv)
    word2vec_features = load_from_csv(word2vec_csv)
    encoded_data = one_hot_encode_vulnerabilities(tfidf_features, word2vec_features)
    train_data, test_data = smote_sampling(encoded_data)
    save_as_csv(train_data, 'train_data.csv')
    save_as_csv(test_data, 'test_data.csv')
end procedure
```

```
procedure TRAIN_MODELS(train_csv)
    train_data = load_from_csv(train_csv)
    best_models =
    for model_name, model in models.items() do
        best_model = hyperparameter_tuning_and_train(model, train_data)
        auc, roc_curve_data = calculate_auc_and_roc(best_model, train_data)
        best_models[model_name] = 'model': best_model, 'AUC': auc, 'ROC Curve': roc_curve_data
    end for
    save_best_models(best_models, 'best_models.pkl')
end procedure
procedure TEST_DEPLOY(input_contract_code, best_models)
    input_features = tokenize_and_vectorize(input_contract_code)
    predictions =
    for model_name, model_data in best_models.items() do
        best_model = model_data['model']
        prediction = predict_vulnerability(best_model, input_features)
        predictions[model_name] = prediction
    end for
    chosen_category = 'Access Control'
    vulnerability_present = check_vulnerability_presence(chosen_category, predictions)
    if vulnerability_present then
        print(f"Vulnerability 'chosen_category' is present.")
    else
        print(f"Vulnerability 'chosen_category' is not present.")
    end if
end procedure
```

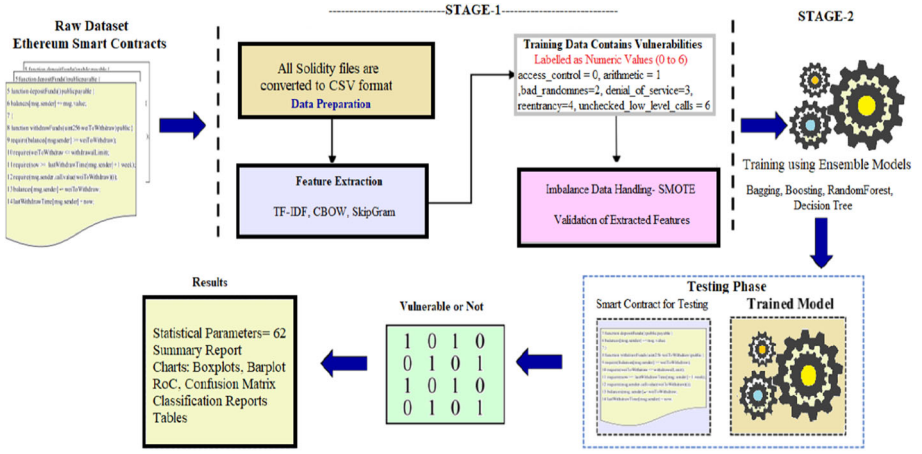



**Fig. 5** Architecture of proposed model

such as Smarti- Audit, we intended to perform feature sensitive vulnerability classification, and therefore we performed different data processing and feature generation methods. Being a machine learning based approach, to train our model, we considered each Smart Contract data (SmartBugs dataset) as one sample, pertaining to a specific kind of vulnerability. Thus, in pre-processing at first, we converted the different Solidity based Smart Contracts into a distinct string type and framed it as one raw of dataset. Similarly, we converted entire Solidity based Smart Contracts as 162 data samples. Subsequently, we performed tokenization of the collected data samples using Alice dictionary. Additionally, some functions like stopping words removal etc. was performed. Noticeably, since the proposed model was a multi-class classification problem, we performed vulnerability labelling of each Solidity based Smart Contracts, where the Smart contracts pertaining to access control, arithmetic, bad randomness, DoS, re-entrancy,unchecked low-level calls, were labelled as 0,1,2,3,4 and 5, respectively. The total vulnerable classes were six.

## 2.2 Data preparation

Data cleaning or preprocessing is also known as "cleaning" text data or removal of artifacts, so that machines can understand it. Preprocessing transforms data into usable form and highlights textual elements that an algorithm can use. This can be accomplished in a variety of ways as described in the succeeding sections.

### 2.2.1 About dataset

Solidity is a high-level language for implementing smart contracts and it is contract-oriented. It is a set of functions and data stored on the Ethereum blockchain at a particular address. These programmes control how accounts behave in the Ethereum state. The program of solidity is written in curly brackets. It is inspired by C++, Python, and JavaScript, and was created with the Ethereum Virtual Machine in mind (EVM). Solidity is statically typed and, among other things, enables inheritance, libraries, and sophisticated user-defined types. The files collected from below mention sources are in .sol format. We used the different types of vulnerabilities

as search terms on Google, and discovered numerous susceptible smart contract repositories, including not-so-smart-contracts and SWC Registry[7]. The latter is a standardized method for Ethereum smart contract security flaws that is mentioned in Mythril's GitHub source[8]. We also collected vulnerabilities from reputable entities in blog articles such as Positive.com[9] and Blockchain.unica[10] where smart contracts are audited, tested, or debated. Finally, we utilised Etherscan to gather smart contracts that have been implemented on the Ethereum network and are proven to be vulnerable. It includes different vulnerabilities, total of 132 files were included for this work. We considered SmartBugs standard dataset from GitHub repository, which contains huge number of required files is for Smart Contract Vulnerability detection. The smart contracts files are programs written in solidity having extension as .sol. To analyze Ethereum smart contract vulnerabilities, we employ the taxonomy described in the DASP[11]. Each contract is assigned to top-10 categories. Six out of ten categories were chosen. However, unlike classical approaches such as SmartiAudit, we intended to perform feature sensitive vulnerability classification, and therefore we performed different data processing and feature generation methods. Being a machine learning based approach, to train our model, we considered each Smart Contract data (SmartBugs dataset) as one sample, pertaining to a specific kind of vulnerability. Thus, in pre-processing at first, we converted the different Solidity based Smart Contracts into a distinct string type and framed it as one row of dataset.

Similarly, we converted entire Solidity based Smart Contracts as 162 data samples. Subsequently, we performed tokenization of the collected data samples using Alice dictionary. Additionally, some functions like stopping words removal etc. was performed. Noticeably, since the proposed model was a multi-class classification problem, we performed vulnerability labelling of each Solidity based Smart Contracts, where the Smart contracts pertaining to access control, arithmetic, bad randomness, DoS, re-entrancy, unchecked low-level calls, were labelled as 0,1,2,3,4 and 5, respectively. The total vulnerable classes were six.

*Preprocessing of data* Preprocessing of data is a crucial stage in the machine learning pipeline. We can't just feed raw data into a model. To build a strong and accurate machine learning model, we need to clean the data and use certain techniques such as

- Conversion from original solidity file format to comma separated file format. *.sol to .csv*.
- Regular Expressions Customized regular expressions are generated to remove the noise from the smart contract dataset such as puntuation marks, numerals, special characters, blank spaces etc
- Tokenization is the act of breaking a sentence's sequence of words into smaller parts called tokens so that it may be read and understood. Words, numbers, and punctuation marks are all examples of tokens. Word segmentation is another name for tokenization.
- Stop words In English language there are certain words which acts like connectors. On their own they cannot give complete meaning until connected with function words, such kind of useless words are no longer needed for processing, even a search engine has been programmed to ignore, these words are removed from the smart contract. One advantage of removing such words fastens the computation as well as meaning of the original sentence will not be lost.

[7] https://swcregistry.io/

[8] https://github.com/skylightcyber/mythril-classic

[9] https://blog.positive.com/

[10] https://blockchain.unica.it/

[11] https://dasp.co/

- Lemmatization we extracted root words from the smart contracts to avoid any ambiguity in understanding of similar words.

### 2.3 Smart contract vulnerabilities

Six different types of vulnerabilities considered for this work is listed below. The smart contract are written in solidity and saved as .sol format files. Solidity is a statically typed curly braces coding language, used for writing smart contracts on the EVM- Ethereum Virtual Machine. Smart contracts are programs that are performed within a peer-to-peer network with no central authority, allowing for the implementation of tokens of value, ownership, voting, as well as other types of logic.

The concept of *Reentrancy* is familiar in computing. Perhaps it is the most well-known Ethereum vulnerability in smart contract arena. The call() function is at the centre of this attack, and it is used to invoke a function in the same contract or another contract to transmit either data / Ethereum. It initiates the program execution and consumes all available Gas, there is no Gas limit until we indicate one. A call to an untrusted contract or the employment of a low-level function with an external address can cause its state to alter in the midst of a function's execution. Set of circumstances of an Reentrancy attack are as follows. With its public withdraw() function, a smart contract can track the balance of a range of external addresses also enable users to obtain funds. The withdraw() function is used by a malevolent smart contract to transfer its whole amount.

**Listing 3** Re-entrancy Vulnerability

```
function Collect(uint _am)
public
payable
{
var acc = Acc[msg.sender];
if( acc.balance>=MinSum && acc.balance>=_am && now>acc.unlockTime)
{  // <yes> <report> REENTRANCY
if(msg.sender.call.value(_am)()) {
acc.balance-=_am;
Log.AddMessage(msg.sender,_am,"Collect"); }
}}
```

Prior to actually updating the infected contract's balance, the victim contract uses the low-level call.value(amount)() function to transmit eth to malicious contract. The malicious contract does have a payable fallback() function which collects the funds before calling the withdraw() function of the victim contract. T the balance amount of the malicious contract hasn't been updated since the initial withdrawal, therefore this second execution initiates a fund transfer. As a result, the fraudulent contract is able to withdraw the entirety of its balance for the second time.

Not just smart contracts, but all applications have access control difficulties. In fact, it ranks fifth among the top ten OWASP vulnerabilities. The functionality of a contract is normally accessed by its public or external functions. Access control skips are more sophisticated than insecure visibility settings, which offer attackers easy access to a contract's secret data or logic. These flaws can take place when contracts employ the deprecated tx.origin to check

callers, handle complex authorization logic with long requires, or use delegate calls in proxy libraries or proxy contracts without thinking.

**Listing 4** Access Control Vulnerability

```
contract Missing{
  address private owner;
  modifier onlyowner {
    require(msg.sender==owner);
    _;  }
  // <yes> <report> ACCESS_CONTROL
  function IamMissing()
  public   {
    owner = msg.sender;  }
  function () payable {}

  function withdraw()
  public
  onlyowner
  { owner.transfer(this.balance); }
}
```

Scenario of an Attack are:- The address that initializes a smart contract is designated as the contract's owner. This is a frequent structure for offering unique rights, such as the power to withdraw monies from a contract. Apparently, anybody can call its initialization function, even if it has already been called. Enabling anybody to become the contract's owner and gain control of its funds.

The contract's initialization function gives the opportunity to any caller of a function as its owner in the following scenario. The logic, on the other hand, is separate from the contract's function and it doesn't keep track of how many times it's been called.

Although integer overflows or underflows aren't really a new sort of vulnerability, they are particularly problematic in smart contracts, since unsigned integers are common and therefore most programmers are used to working with simple int types. Many seemingly innocent codepaths becomes vectors for loss of service if overflows occur.

**Listing 5** IntegerOverflow Vulnerability

```
contract IntegerOverflowMul{
  uint public count = 2;
  function run(uint256 input) public {
    // <yes> <report> ARITHMETIC
    count *= input;
  } }
```

The withdraw() function of a smart contract permits users to retrieve ether provided to the contract if balance remains positive following the recent operation. An assailant tries to extract more money than he or she has. The outcome of the withdraw() function test is always positive, this allows attacker to withdraw more than permitted. As a result, the balance falls below zero and is a substantially higher than it should be.

Despite the fact that the standard threat model handles the situation of a miner, the majority of block related pseudo-random features can be abused far more readily. The EVM's notable block-related attributes are block.coinbase , difficulty, gaslimit, number, timestamp, and blockhash. For all of them, an attacker can obtain the same facts as the victim contract by simply executing a transaction in same block as the victim contract.

**Listing 6** Bad Randomness Vulnerability

```
function settle() public {
require(block.number > guesses[msg.sender].block);
// <yes> <report> BAD_RANDOMNESS
bytes32 answer = blockhash(guesses[msg.sender].block);

guesses[msg.sender].block = 0;
if (guesses[msg.sender].guess == answer) {
msg.sender.transfer(2 ether);
} }
```

While it appears that we have a $1/2^{256}$ chance of guessing the correct hash, blockhash really returns zero for block numbers which more than 256 blocks, so attacker can guess zero or wait.

In the Ethereum world, denial of service is a disaster. Denials of service can occur in a variety of ways, such as maliciously acting as the receiver of a transaction, artificially raising the gas required to process a function, misusing access controls to gain access to sensitive components of smart contracts, or carelessness.

**Listing 7** Deneil of Service Vulnerability

```
contract DosOneFunc {
address[] listAddresses;
function ifillArray() public returns (bool){
if(listAddresses.length<1500) {
// <yes> <report> DENIAL_OF_SERVICE
for(uint i=0;i<350;i++) {
listAddresses.push(msg.sender);}
return true;
} else {
listAddresses = new address[](0);
return false; } }
}
```

When smart contracts are launched or functions within them are invoked, amount of gas required to execute these activities is always determined by amount of computations required. The Ethereum platform has a block gas restriction, which must not be exceeded by the total of all transaction in a block. Whenever the cost of executing a procedure surpasses the block gas limit, coding patterns that seem to be harmless in centralized applications might cause Denial of Service problems in smart contracts. DOS problem can be caused by altering an array of unspecified size that grows in size over time.

The low-level procedures transmit(),call(), delegatecall(), or callcode() are among of Solidity's more interesting features (). Their error-handling behaviour differs from that of other Solidity functions in that; they do not propagate and do not result in a complete reversion of the present execution. Instead, they'll return a misleading boolean value, and the programme will continue to function. This can catch developers off guard, however if the return value of these kind of low-level calls isn't checked, it can result in fail-opens as well as other undesirable results.

**Listing 8** Unchecked Low Level Calls

```
function withdrawal(uint256 _amount1) public {
requires(balances[msg1.sender] >= _amount);
balanceLeft[msg1.sender] -= _amount;
etherLeft -= _amount;
msg1.sender.send(_amount);
}
```

The code above illustrates what can go completely off the mark if a return value of transmit is not checked before send(). The Ethereum Virtual Machine will substitute its return value as untrue if the call was being used to deliver ether toward a smart contract which does not accept it (for example, since it lacks payable fallback mechanism). The method's alterations to the contracts' state will not be undone because the return value has not been checked in this instance, and the etherLeft parameter will end up keeping an inaccurate value.

## 2.4 Feature engineering strategies used for proposed system

Advanced Feature Extraction techniques as displayed in Fig. 6 are used for handling text data and extracting meaningful features from the same, which are used in stage-2 machine learning system of the proposed model. Once labelling the Solidity based Smart Contracts, we applied three different kinds of feature extraction algorithms. We applied TF-IDF, CBOW,Skip-N Gram methods as shown in Fig. 6, which resulted three different feature sets.

As computers recognize binary language, which is made up of 0s and 1s. It's impossible to make them automatically understand any natural language. However, we may address our problem by encoding such phrases into numeric form. Vectorization is the process of turning textual information into numbers. Sparse Vector Representations and Dense Vector Representations are two methods for converting text to numbers. The vector space paradigm, in which each document is represented as a vector, is well-known in information retrieval. The weights or significance of each word in the text are represented by the vector components. The cosine similarity metric is used to determine how similar two text documents are. Author [77] developed the Word2vec method, which employs a huge quantity of text to build high-dimensional (50 to 300 dimensional) representations of words without the usage of external annotations. Many language regularities appear to be captured by this depiction.

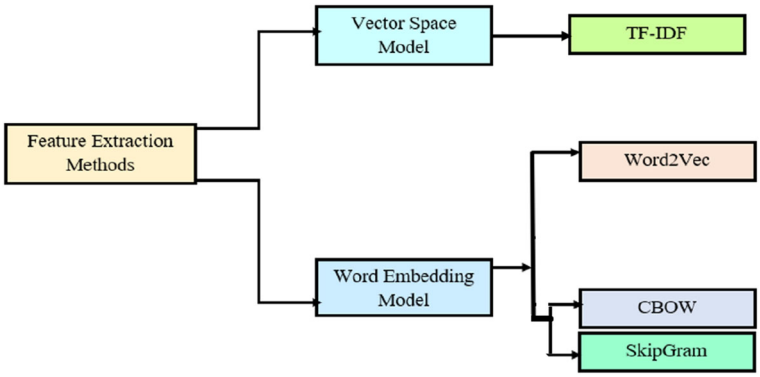



**Fig. 6** Different featureset were extracted from the Dataset

### 2.4.1 TF-IDF model

One of the most often used word weighting techniques in information retrieval systems is Term frequency–inverse document frequency (TF-IDF). Despite its ubiquity, TF-IDF is sometimes seen as an empirical approach, particularly from a probabilistic standpoint, with several variants. The TF-IDF is indeed a numerical statistic which shows the importance of a word in a document based on the dataset it belongs to by scoring the words in the text. For word and keyword relevant extraction, this technique has been examined in numerous studies. This model generated 3712 columns and 130 rows. The objective function in TF-IDF is the negative log-likelihood of a word given a set of context i.e -log( p( wo/wi )), where p( wo/wi ) is given as:

$$tfidf_{i,d} = tf_{i,d} \cdot idf_i \ldots\ldots\ldots\ldots\ldots\ldots\ldots\ldots\ldots\ldots (1)$$

One of the most significant and commonly utilised ideas in information retrieval is inverse document frequency (IDF) [78]. It is used in conjunction with the word "frequency" (TF). As a result, a very successful word weighting approach for retrieving information [79] has been developed. The TF-IDF method is a quick and easy way to match terms in a query to documents that are relevant to that query. TF-IDF returns texts which are highly relevant to a query based on the data obtained. If a user enters a query for a certain topic, TF-IDF can locate documents that provide information related to the query. Furthermore, TF-IDF encoding is simple, making it appropriate for use as the foundation for more complex algorithms and query retrieval systems.

### 2.4.2 Word-to-vector model

The word2vec tool accepts a dataset as input and outputs word vectors. It learns vector representations of words after constructing an vocabulary from the training text input. Many NLP and ML programs utilize the resultant word vector file as a feature. Semantic connection refers to the act of keeping related words together while keeping dissimilar words separate. Word2vec reconstructs the words' linguistic context. Finding the nearest words for a user-specified term is a straightforward approach to explore the learnt representations. This is what the distance tool is for. Continuous bag-of-words and continuous skip-gram are the two major learning methods in word2vec. The user can select one of these learning algorithms using the switch CBOW. Both methods learn a word's representation, which is helpful for forecasting of other words in the sentence[12]. Word2Vec uses two different learning models: Continuous Bag of Words (CBOW) and Continuous Skip Gram (CSG) (skip-gram), both are neural network models. Word2vec employs a fully or completely connected network with one hidden layer. Linear neurons make up the entirety of the hidden layer. The neuron count in the input layer is equal to the number of words in the training vocabulary. The dimensionality of the generated word vectors determines the hidden layer size. The output layer is identical to the input layer in terms of size. Assuming a vocabulary of V words for learning word vectors and a dimension of N for word vectors, the input to hidden neuron layer links may be represented by a matrix $W * I$ of size $V x N$, with each row representing a vocabulary word. Similarly, a matrix WO of dimension $N * V$ may be used to express the relationships between the hidden and output layers. In this example, each WO matrix column represents word from dictionary. The network's input is encoded using the "1-out of -V" encoding, which means only single input line is assigned to one and the remaining are adjusted to zero. We consider pretrained

[12] https://iksinc.online/tag/continuous-bag-of-words-cbow/

models from Google News and Glove from stanford along with the Smart contract dataset, by keeping in mind that the future query can be different from the one avilable in dataset, any new combination of smart contracts can be provided as test query, we can expect accurate result from the proposed system. There are 100 billion words in the Google News dataset. Comprising 300 dimensional vectors in the model for 3 million words and phrases. Glove is an unsupervised method of creating word vector representations. The training is based on a corpus's aggregated global word-by-word co-occurance data. Use of such models have the ability to improve the NLP model's performance. This model generated 300 columns and 19663 rows for 33758 rows of dataset. Since the model is based on artificial neural network, back propagation or gradient based learning or optimization, there is no; one to one matching of words, the model is highly probabilistic in nature which captures context or objective of the sentence, in terms of meaningful syntactic and semantic regularities. The resultant features are saved in csv and pkl file format. Pickle is a Python module that serialises objects so that they may be saved to a file and reloaded when the application calls them. Pickling and unpickling are terms used to describe the process of serialising and deserializing files. Pickling is done in order to conserve space when storing or transmitting a file. The Pickle module serialises files & stores them as.pkl.

#### 2.4.3 Continuous bag of words model

Bag of Words(BOW) is an approach for dealing with words and context in text processing or information retrieval. It refers to a way in which a group of words are represented without retaining order.

The CBOW architecture model tries to predict the current target word or center word based on the source context word, there is no one to one matching of the input to generated vector, it is not easy to understand the result of complex neural net model by human, but machine can understand and extract the contextually accurate and relevant features from the raw data. Feature selection implies carefully selecting prominent features from the entire feature-set. Another important aspect of feature selection is dimensional reduction, which means reducing the number of features. Dimensionality reduction is useful technique, when we have computationally high-dimensional data. This model generated the 100 dimensional feature vector, values of the vector are shown below.

#### 2.4.4 Skip-Gram model

A skip-gram model is a dense approach of creating a word vectors using the neural Network. The aim of the neural network in this case, is to predict contextual or neighboring words,from a word. The skip-gram model swaps the order in which target or context words are used. The target word is supplied at the beginning, while hidden layer remains unchanged, and its NN output layer is duplicated numerous times to fit the desired amount of context words.

The Skip-gram approach is most effective for huge data sets because each context-target pair is treated as a new data point. By calculating the correlation with dot product, the SGK model calculates the probability of the centre word occurring with every context words, and then translates the similarities into probability by running it via soft-max functions. The Skipgram model also resulted in 100 columns and 130 rows of feature set.

The dispersed representations of context are integrated in the CBOW model to forecast the word. Here the word belongs to smart contract fundamental units. The distributed representation of the input word is utilized to forecast the context in the Skip-gram model as shown in Fig. 7. Hence both CBOW and SKG features are treated as single feature.

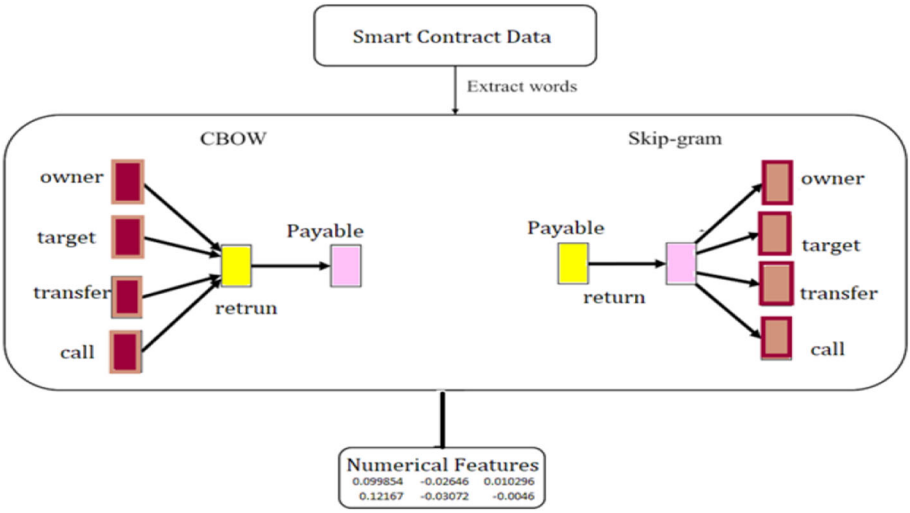



**Fig. 7** Bag of words and Skipgram model resulted in one featureset

## 2.5 Imbalance handling

Realizing the fact that though, the use of Word2Vec feature extraction methods as stated above retrieves most significant and low-dimensional semantic features to perform classification. however, the classification accuracy with original features were found inferior. This was because of the data imbalance problem, as the data elements pertaining to the different types of vulnerabilities were different and many were having minority class, while a few were in majority class. To alleviate such class imbalance or data imbalance problem, we applied Synthetic Minority Over Sampling Method, often called SMOTE sampling. Unlike classical Up-sampling and Down-Sampling methods, the proposed SMOTE model avoids any biasing effect and hence possesses uniform sample distribution. This as a result helps training better and hence higher accurate.

Imbalanced feature sets obtained from previous steps are a type of classification issue in which the distribution of classes is not uniform. They are usually divided into two classes, the majority class and the minority class as shown in Fig. 8. There are various methods for dealing with a dataset's imbalance. Major categories are resampling and Algorithmic. Before

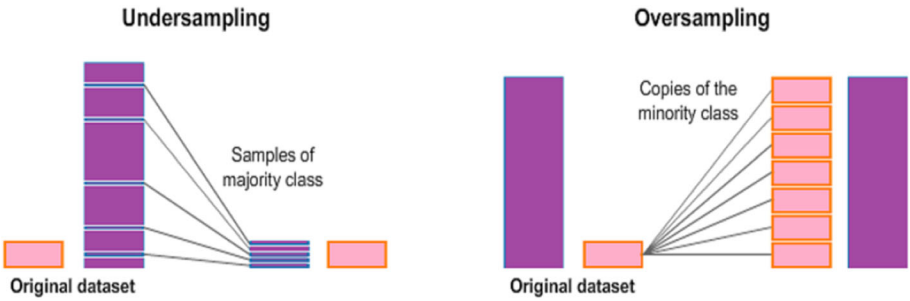



**Fig. 8** Sampling results

**Table 5** Average output values of 36 Models

| Precision | Fmeasure | Recall | AUC | Accuracy |
|---|---|---|---|---|
| 0.600613 | 0.844903 | 0.589995 | 0.878935 | 0.858329 |
| 0.745558 | 0.916163 | 0.636653 | 0.903792 | 0.922604 |
| 0.788678 | 0.920447 | 0.812169 | 0.86113 | 0.920735 |
| 0.782975 | 0.919931 | 0.787558 | 0.88089 | 0.921831 |
| 0.792867 | 0.922255 | 0.783239 | 0.880465 | 0.921567 |

Highest values were shown in red color

passing the data to the machine learning algorithm, we focus on balancing the classes in the data preparation or feature engineering. The basic goal of class balancing is to either raise the minority class's frequency or reduce the dominant class's frequency. This is done to ensure that each classes have roughly the same number of instances. Re-sampling is a data-level technique, it is most widely used and popular approach. Under or down sampling. By removing majority class instances at random, down-sampling tries to balance class distribution. This continues until the majority and minority class instances are evenly distributed.Over or Up-sampling extends the number of instances in the minority class by reproducing them at random in order to offer a more representative sample of the minority class as shown in Figure 8.

In general, we are most interested in the minority class, and we strive to get the best outcomes in this class. The efficiency of the classifier model will be harmed if imbalanced records are not addressed beforehand. The significant proportion of the predictions will be for the majority class, and thus the minority class features are considered as unwanted or noisy data, hence ignored. SMOTE is the oversampling approach in which artificially or synthetic generated data for the minority class were added. This approach aids in overcoming the problem of overfitting caused by random oversampling. It concentrates on the feature space in order to produce new examples by interpolating between positive samples that are close together. To be noted, since in this research we formulated our model in such manner, that it assesses whether a specific Smart Contract is of X-type vulnerability, and therefore before SMOTE sampling, we mapped input three features (TF-IDF, CBOW and Skip-N Gram) as 36 feature sets. Thus, with a total of 36 feature-sets, performing SMOTE sampling, we achieved a total of 130 feature sets, which were later used for model training. Based on experiment conducted, all five classifier models were unable to generalize adequately on the minority class when compared to the majority class on the original imbalanced data. As a result, the majority of the negative class samples were classified properly. As a result, there were fewer FP and more FN. On the test data, there is a notable increase in Recall after oversampling. A comparison table for all five models is presented in Table 5 to help understand this better :

### 2.5.1 Feature validation

In this re-sampling technique, the whole data is divided into k sets of almost equal sizes, we have chosen 5 fold. The first set is selected as the test set and the model is trained on the remaining k-1 sets. The error rate is measures once the model is fitted to test dataset (Fig. 9). In the second pass, the procedure is repeated with k-1 set. Obtained error for all passes were accumulated for further processing, the interesting aspect of this type of validation is no sample is missed or left, every set will be included exactly one time. When we change the k value from lower to higher, we are changing the number of folds, more folds more

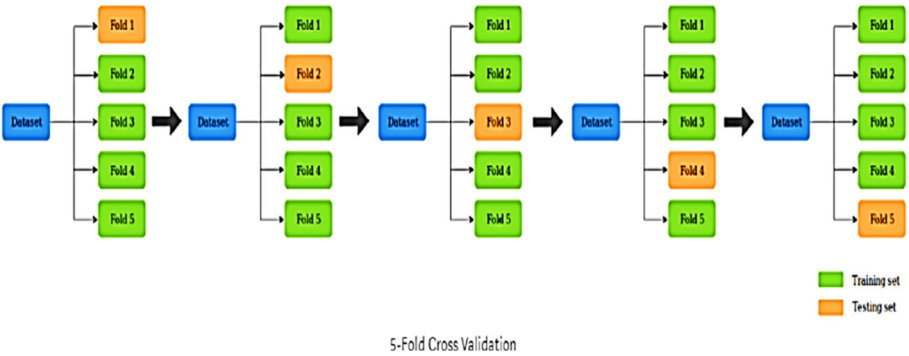



**Fig. 9** 5Fold cross validation

better the model is and provides low variance. As bias increases variance decreases both are inversely proportional, hence a good balance or trade off has to be obtained so that data will neither be under fitted nor over fitted. It is essential for the application or task to carryout validation of features using k-fold technique to get more appropriate and accurate results from the ML model. Only demerit of this approach is expensive in terms of computation because the model has to repeat k times from the scratch.

## 2.6 Stage-2

Ensemble learning supports the concept of "wisdom of crowds," which says that more number of algorithms might make better decisions than a single expert or efficient technique. Ensemble learning, on the other hand, belongs to a collection (or ensemble) learners, or models, which work together to improve the final prediction. Due to significant variation or bias, a single model, typically referred as a base or weak learner, might not always perform effectively on its own. When weak learners are combined, however, they can generate a strong learner, as their combined bias and variance are reduced, resulting in improved model performance (Fig. 10).

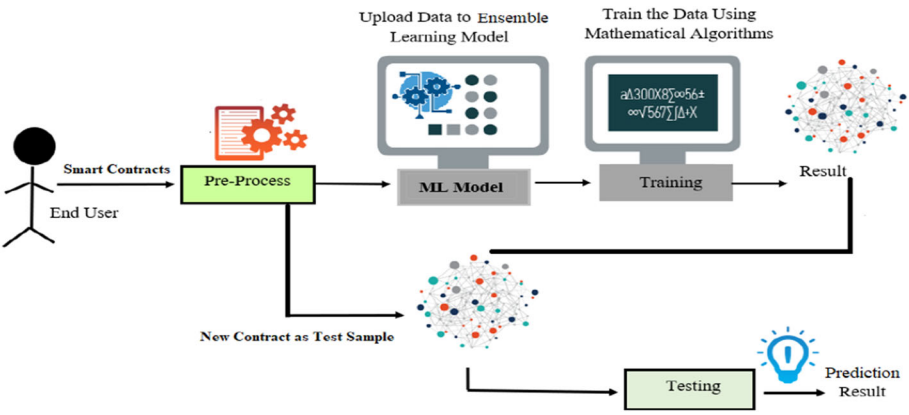



**Fig. 10** Ensemble model for multi-class vulnerability classification

Once obtaining a total set of 130 feature sets, we applied different machine learning algorithms to perform multi-class (vulnerability) classification. More precisely, we applied the following machine learning models: 1. Decision Tree 2. Random Forest 3. Bagging 4. AdaBoost 5. Gradient Boost Noticeably, these algorithms were applied distinctly to perform multi-class classification with 5-fold cross-validation. Thus, these algorithms classified each Solidity based Smart Contracts as Access control, Arithmetic , bad-randomness, Denial of Services (DoS), re-entrancy, and unchecked low-level calls. Data obtained after feature extraction are combined, then passed for handling imbalance nature of extracted features. 5 fold cross validation technique is applied to obtain the processed features, since we are using five different algorithms on the feature engineered data, it results into 5 algorithms X 6 types of vulnerabilities = 18 different records for presence of vulnerability and 18 for benign records, thus resulting 36 different models. each of this model is passed for training, the intermediate results are saved as predicted files with and without probabilities having 11 dimensional data, 10 features and one target. The models are saved pickle file, and result as NPZ files. This helps in reducing the computation time significantly for training phase. We also obtained the best features and training model. During testing phase these saved files are used, a test sample (smart contract in .sol format) is accepted from the user then sent for feature engineering phase, and trained model is employed to forecast the vulnerability class. The results of processing are obtained in the form of plots and tables as mention in the result section. A confusion matrix is a summarization of classification problem on prediction outcomes. The number of right and unsuccessful predictions is totaled and broken down by class using count values. In the following section how each model has performed the classification of vulnerabilities can be seen. Two confusion matrix are generated after extenuation of the algorithms, (1) based on combined features of TFIDF, W2VEC, CBOW, and SKIPGRAM the 6 different vulnerabilities were given to the each ensemble model, which identifies presence and absence of vulnerbility in the smart contract, 0-means No-Vulnerability and 1-means Presence of Vulnerability. (2) based on only one type of Feature extraction-TFIDF and one algorithm at a time.

### 2.6.1 Decision tree (DT)

Decision tree algorithms are well-known machine learning approaches that have been applied to a variety of issues, including classification challenges. For classifying datasets, decision trees give a nonparametric technique. Mathematically speaking, a DT has low bias and high variance.Because of their capacity to convert vast complex data into incredibly simple yet information-rich graphs and charts, decision trees was selected for this research. By considering the mean results of many decision trees may reduces the variance issue, while maintaining that low bias. Combining trees is known as ensemble method. The resulting graphical tree representation was thought to be a valuable tool for quickly elucidating the crucial parameter combinations that lead to unacceptable losses, which can then be turned into a series of rules. Additional benefits of employing decision tree algorithms include minimum data preparation demands and reliable performance on huge datasets. Confusion matrix for DT model is shown in Fig. 11 and normalized is given in Fig. 12.

### 2.6.2 Random forest

Random forest is a supervised classifier that works better for labelled data. It creates a "forest" from an ensemble models of decision trees, which are commonly trained using the "bagging"

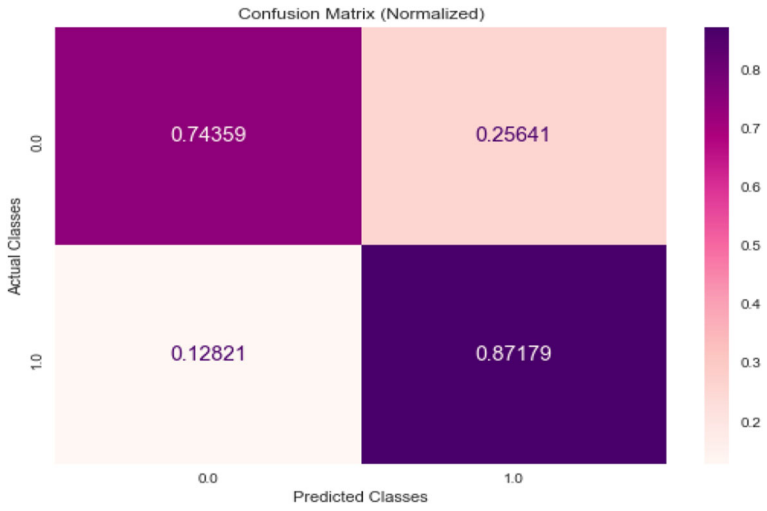



**Fig. 11** DT model

method. The bagging method's basic idea is to combine many learning models to improve the overall output. Random forest is a versatile, simple machine learning technique, in most cases, gives excellent results even without hyper-parameter adjustment. For its clarity and versatility, it is also considered as most widely used algorithms. Confusion matrix for RF model is shown in Fig. 13 and normalized is given in Fig. 14.

### 2.6.3 Bagging

The ensemble learning method of bagging, also referred as bootstrap aggregation, is commonly used to minimize variance within a noisy population. This algorithm is introduced

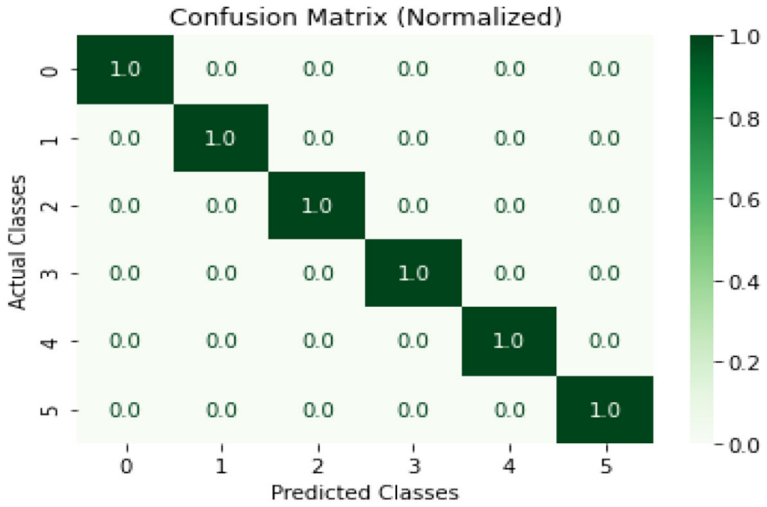



**Fig. 12** All 6 classes of vulnerability

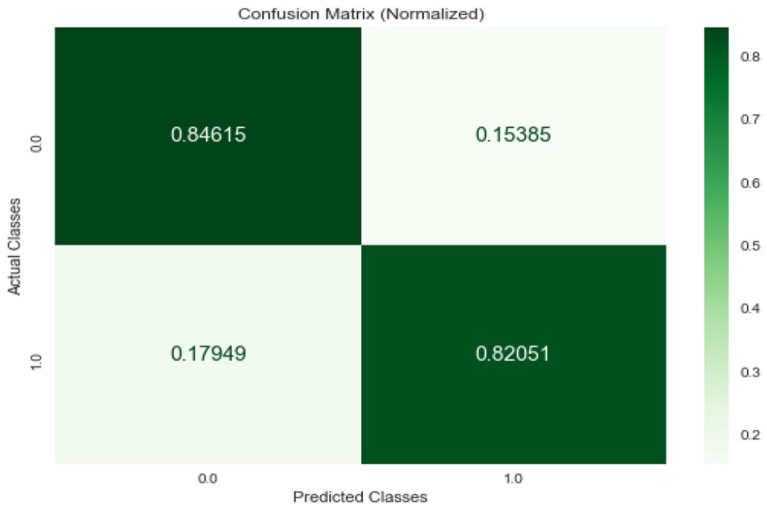



**Fig. 13** RF model

by [80]. Bagging process selects randomized sample of data from a training examples with replacement; that is, the single data points might be chosen many times. Following the generation of multiple data samples, these weak model can be trained individually, & based on type of operation regression or classification, the average or majority of those predictions, for example, produce a more accurate assessment. Weak learners are taught in parallel in bagging, but sequentially in boosting. Confusion matrix for Bagging model is shown in Fig. 15 and normalized is given in Fig. 16.

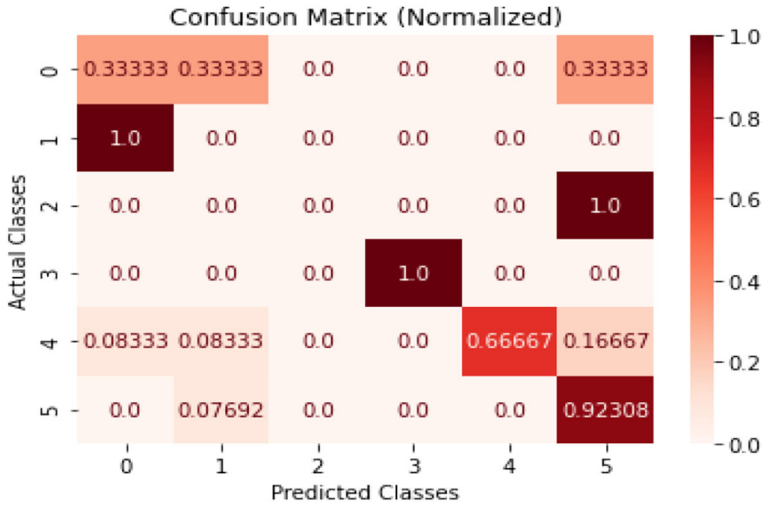



**Fig. 14** All 6 classes of vulnerability

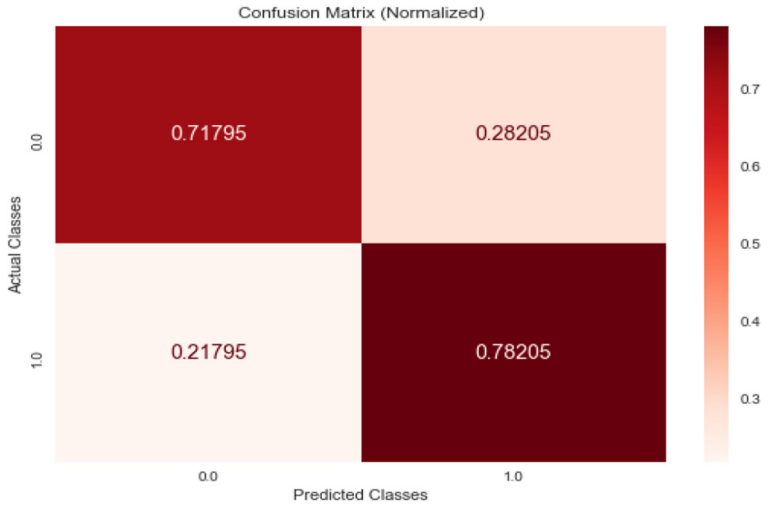



**Fig. 15** Bagging model

### 2.6.4 AdaBoost

hat aims at combining multiple weak classifiers to build one strong classifier AdaBoost, also known as Adaptive Boosting, combines weak classifiers to develop one stronger classifier that is employed as part of an Ensemble Method. This model especially well with DT. DT with one level, or DTs with only one split, are the most popular algorithm used with AdaBoost. Decision Stumps is another name for these trees. This approach creates a model by assigning equal weights to all of the data points. It then gives miss classified points which are incorrectly labeled a higher weight. In the next model, all points with greater weights are given more

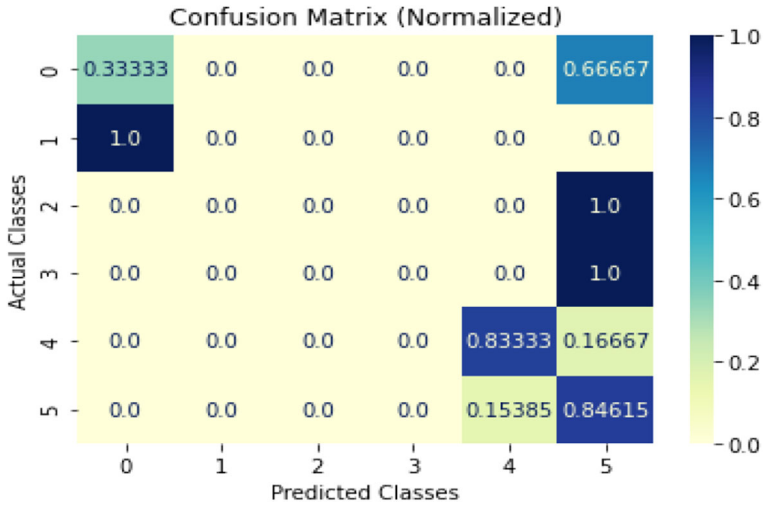



**Fig. 16** All 6 classes of vulnerability

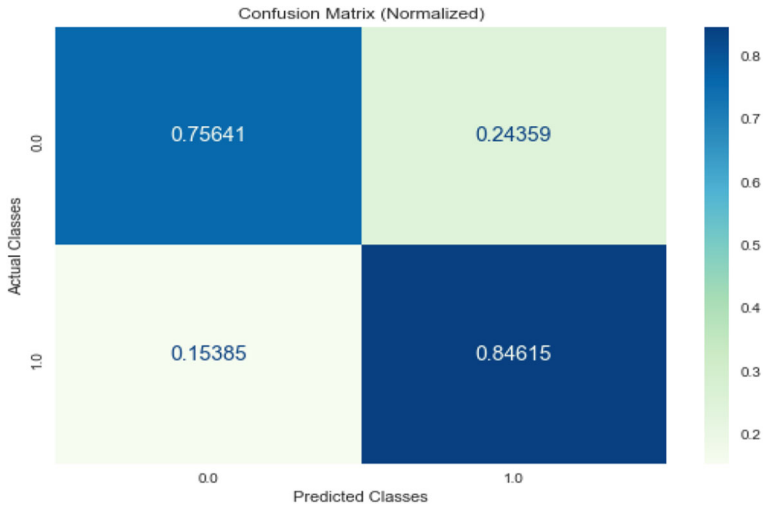



**Fig. 17** Adaboost model

importance. It will continue to train models till a lower error is received. Confusion matrix for AdaBoost model is shown in Fig. 17 and normalized is given in Fig. 18.

### 2.6.5 Gradient boost

Gradient boosting recasts boosting as a numeric optimization problem in which the goal is to reduce the algorithm loss function by using gradient descent to add weak learners. Gradient descent is a first-order iterative optimization process for locating a differentiable function's minima. Gradient boosting is a flexible approach to regression, multi-class classification, and

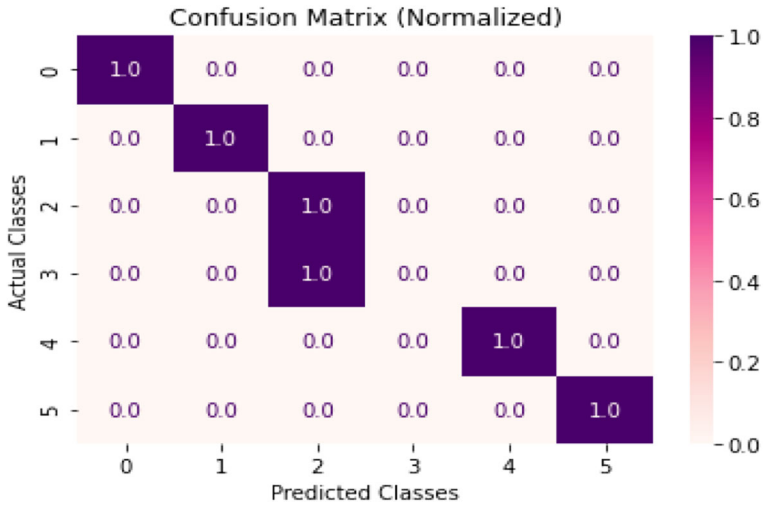



**Fig. 18** All 6 classes of vulnerability

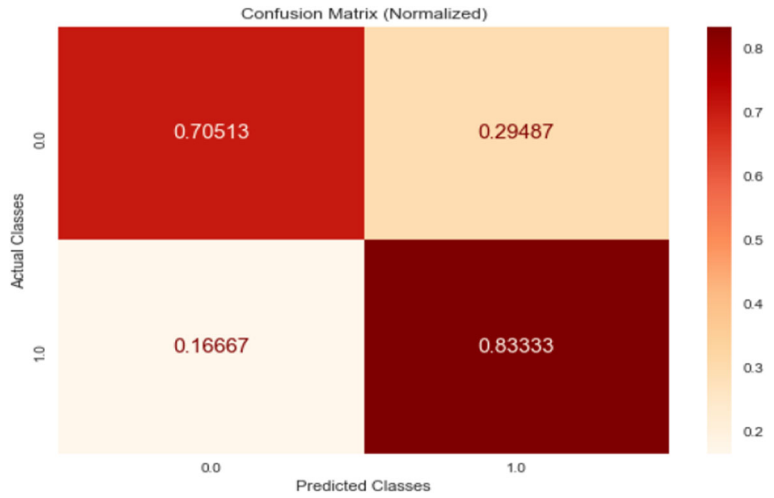



**Fig. 19** Gradient boost model

other applications because it is based on significantly reducing a loss function. Confusion matrix for Gradient boost model is shown in Fig. 19 and normalized is given in Fig. 20.

## 3 Results and discussions

When it comes to tackling classification problems, the confusion matrix is a prominent tool. It can be used to solve multi-class and also binary classification problems.Confusion matrices are used to depict counts based on expected and actual values. The output "TN" stands for True Negative, and it displays the number of correctly identified negative cases.

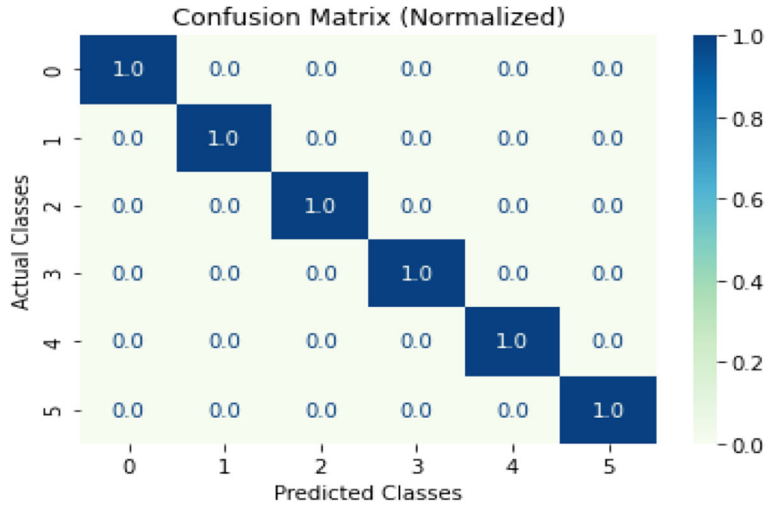



**Fig. 20** All 6 classes of vulnerability

Similarly, "TP" stands for True Positive, which denotes the number of correctly identified positive examples. The terms "FP" and "FN" stand for False Positive and False Negative values, respectively. "FP" stands for False Positive value, which is the number of actual negative examples classified as positive, and "FN" stands for False Negative value, which is the number of actual positive examples classified as negative. Accuracy is one of the most widely utilised criteria while performing categorization. The following formula is used to calculate the accuracy of a model (via a confusion matrix).

$$ACC(Accuracy) = \frac{TP + TN}{P + N} = \frac{TP + TN}{TP + TN + FP + FN} \tag{1}$$

Only three of the four parts of a confusion matrix were utilised in F-measures, therefore two classifiers with distinct TNR scores also have the same F-score. As a result, the AGF metric is created, which makes use of all parts of the confusion matrix and gives samples that are correctly identified in the minority class greater weight (Fig. 21).

$$AGF(AdjustedF - score) = \sqrt{F_2 \times InvF_{0.5}} \tag{2}$$

The AGM (Adjusted geometric mean) of specificity and sensitivity has been represented as modified version.

$$N_n = \frac{N}{POP} \tag{3}$$

$$AGM = \frac{GM + TNR \times N_n}{1 + N_n} \tag{4}$$

$$AGM = 0; TPR = 0 \tag{5}$$

AM (Automatic/Manual):The divergence between automatic and manual classification, that is, the distinction between positive outcomes and positive samples

$$AM = TOPP - (TP + FP) - (TP + FN) \tag{6}$$

AUCI: Area under the curve extrapolated to infinity. The values shown below indicate the performance of an algorithm.

Precision or positive predictive value (PPV): The F1 score (also F-score or F-measure) is a metric of a test's accuracy in statistical classification evaluation. To compute the score, it takes into account both the precision p and the recall r of the test. The F1 score is the harmonic

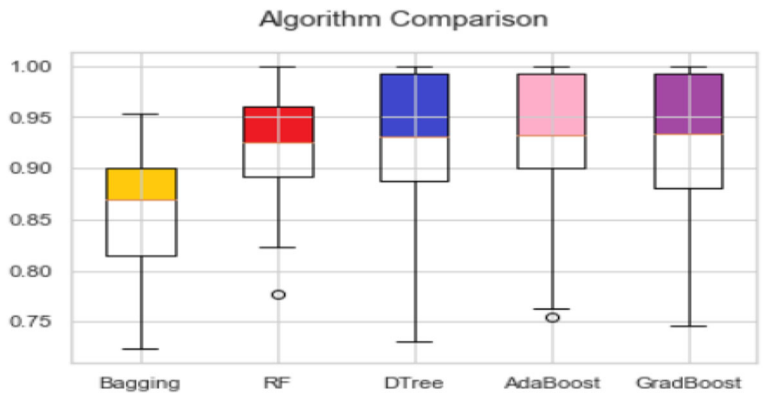



**Fig. 21** Accuracy

**Fig. 22** Average Accuracy of 130 smart contracts in detecting the vulnerabilities by 5 models

| Overall MCC | Strength of Association |
|---|---|
| 0.3 > | Negligible |
| 0.3 - 0.5 | Weak |
| 0.5 - 0.7 | Moderate |
| 0.7 - 0.9 | Strong |
| 0.9 - 1.0 | Very Strong |

mean of precision and memory, with 1 (perfect precision and recall) being the highest and 0 being the worst (Fig. 22).

$$F_\beta - (1+\beta^2) \times \frac{PPV \times TPR}{(\beta^2 \times PPV) + TPR} \tag{7}$$

Recall or TPR (True positive rate):In various domains, sensitivity (also known as true positive rate, recall, or likelihood of detection) refers to the percentage of positives that are accurately detected.

$$TPR = \frac{TP}{P} = \frac{TP}{TP + FN} \tag{8}$$

Cohen's Kappa statistic is a very valuable metric that is underutilised. A multi-class classification problem arises from time to time in machine learning. In some circumstances, metrics like accuracy or precision/recall don't give us a whole view of our classifier's performance. Kappa is a statistic that assesses qualitative (categorical) item inter-rater agreement. It is typically believed to be a more reliable statistic than a simple % agreement estimate, because kappa considers the possibility that the agreement occurred by chance.

$$Kappa = \frac{ACCO_{verall} RACCO_{verall}}{1RACC_{Overall}} \tag{9}$$

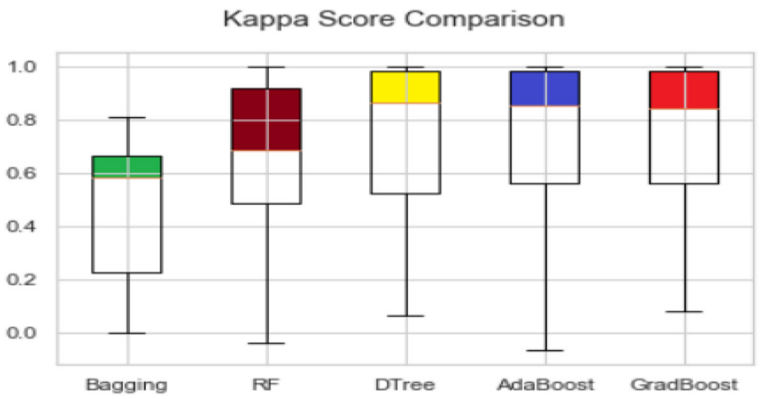



**Fig. 23** Kappa metric obtained from the proposed model

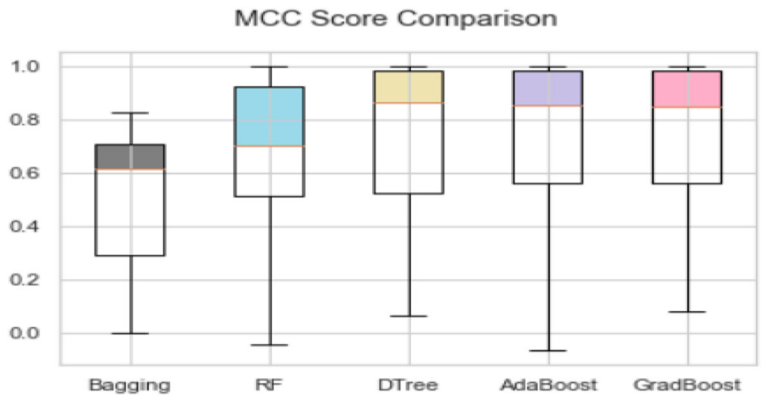



**Fig. 24** Matthews correlation coefficient obtained from the proposed model

The kappa is always less than or equal to one. The classifier is useless if it has a value of 0 or less. Its values cannot be interpreted in a uniform fashion. Landis and Koch propose a method for describing values. A number of 0 indicates no agreement, 0.21–0.40 is fair, 0.41–0.60 is moderate, 0.61–0.80 is substantial, and 0.81–1 is virtually perfect agreement, according to their scheme (Fig. 23).

Matthews correlation coefficient (MCC)- Matthews' benchmark or SOA6: The Pearson product-moment correlation coefficient (not to be confused with Pearson's C) is calculated using a confusion matrix approach. As a result, it has the same meaning.

MCC appears to be the best option for measuring and improving the overall accuracy of the classification model because it is not only easy to interpret but also flexible in the prediction purpose (Fig. 24).

in fact, the correlation between the true classes and the predicted labels is measured. MCC ranges from -1 to +1. A perfect model has a score of +1, while a mediocre model has a score of -1. This attribute is one of MCC's most useful features since it allows for easy interpretation (Fig. 25).

AUC(Area under the ROC curve): The likelihood that a classifier would rate a randomly selected positive sample greater than a randomly picked negative occurrence is equal to the area under the curve. As a result, AUC is the arithmetic mean of each class's sensitivity and

**Fig. 25** Accuracy

| Overall MCC | Strength of Association |
|---|---|
| 0.3 > | Negligible |
| 0.3 - 0.5 | Weak |
| 0.5 - 0.7 | Moderate |
| 0.7 - 0.9 | Strong |
| 0.9 - 1.0 | Very Strong |

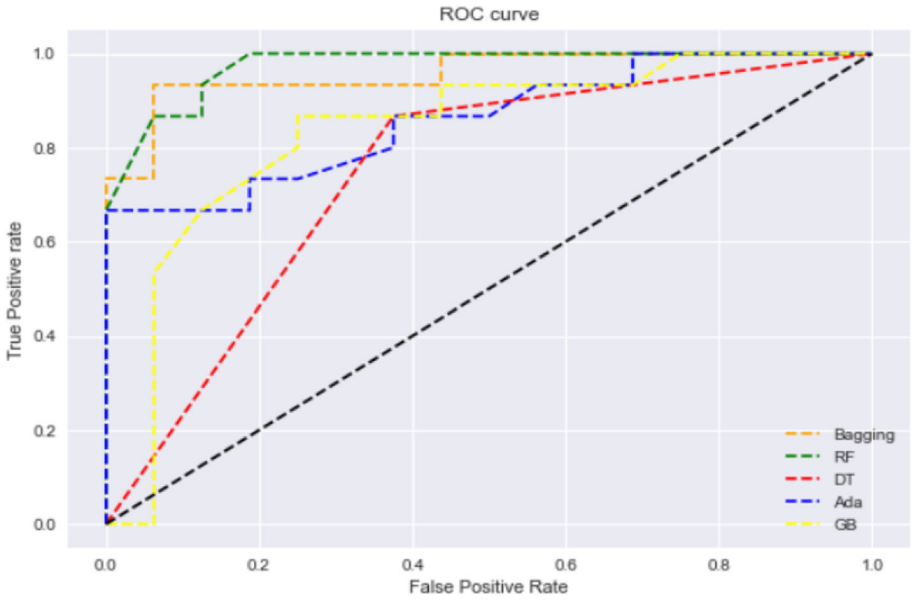



**Fig. 26** ROC

specificity scores.

$$AUC = \frac{TNR + TPR}{2} \tag{10}$$

Our model predicts probabilities for each category and then utilizes a user defined function to convert them into binary labels with a default threshold of 0.5, as shown in Fig. 26. However, other thresholds can be used to interpret probability. Changing this setting can result in a performance improvement. When the cost of one error outweighs the cost of other types of errors, this strategy is useful. Figure 26 shows the TPR as a function of FPR . The trade-off between both rates for a predictive model using different probability thresholds for 5 models is depicted by this ROC curve. This graph also shows our model's strong learning behavior when the discrimination threshold is altered. A ideal model would be represented by a line that runs from the bottom left of the plot to the top left, then across the top to the top right in this sort of plot. The diagonal black line would reflect a model with no learning ability. As can be observed, our models exhibit the characteristics of a skilled model. These curves demonstrate that our model can comprehend the data and learn statistical patterns to distinguish lines with vulnerabilities from those that aren't (Table 6).

## 4 Conclusion

In this paper we have successfully demonstrated the effectiveness of Natural Language Processing (NLP) and Machine Learning (ML) techniques in the detection of vulnerabilities in smart contracts. The security of blockchain systems, particularly Ethereum-based smart contracts, is of paramount importance, and our paper addresses this critical need by leveraging advanced AI technologies. Identification of vulnerabilities in smart contract is an essential as it deals with cryptocurrency. Technical improvements in this area may have a direct

**Table 6** Overall statistics of proposed ensemble model

| Algo | Bagging | | RandomForest | | DecisionTree | | AdaBoost | | GradientBoost | |
|---|---|---|---|---|---|---|---|---|---|---|
| Class | 0 | 1 | 0 | 1 | 0 | 1 | 0 | 1 | 0 | 1 |
| ACC | 0.75 | 0.75 | 0.83333 | 0.83333 | 0.80769 | 0.80769 | 0.80128 | 0.80128 | 0.76923 | 0.76923 |
| AGF | 0.73554 | 0.76442 | 0.83976 | 0.82694 | 0.77681 | 0.83922 | 0.7798 | 0.82304 | 0.73957 | 0.79899 |
| AGM | 0.76023 | 0.73886 | 0.82899 | 0.83754 | 0.82736 | 0.78463 | 0.8154 | 0.78549 | 0.78881 | 0.74608 |
| AM | -5 | 5 | 2 | -2 | -10 | 10 | -7 | 7 | -10 | 10 |
| AUC | 0.75 | 0.75 | 0.83333 | 0.83333 | 0.80769 | 0.80769 | 0.80128 | 0.80128 | 0.76923 | 0.76923 |
| AUCI | Good | Good | Very Good | Very Good | Very Good | Very Good | Very Good | Very Good | Good | Good |
| AUPR | 0.74254 | 0.7585 | 0.83558 | 0.83131 | 0.79827 | 0.82226 | 0.7937 | 0.81131 | 0.75698 | 0.78598 |
| BCD | 0.01603 | 0.01603 | 0.00641 | 0.00641 | 0.03205 | 0.03205 | 0.02244 | 0.02244 | 0.03205 | 0.03205 |
| BM | 0.5 | 0.5 | 0.66667 | 0.66667 | 0.61538 | 0.61538 | 0.60256 | 0.60256 | 0.53846 | 0.53846 |
| CEN | 0.75963 | 0.73485 | 0.59225 | 0.60139 | 0.65779 | 0.61201 | 0.67157 | 0.63853 | 0.73073 | 0.68285 |
| DOR | 9.13369 | 9.13369 | 25.14286 | 25.14286 | 19.72 | 19.72 | 17.07895 | 17.07895 | 11.95652 | 11.95652 |
| DP | 0.52963 | 0.52963 | 0.77209 | 0.77209 | 0.71392 | 0.71392 | 0.67949 | 0.67949 | 0.59411 | 0.59411 |
| DPI | Poor | Poor | Poor | Poor | Poor | Poor | Poor | Poor | Poor | Poor |
| ERR | 0.25 | 0.25 | 0.16667 | 0.16667 | 0.19231 | 0.19231 | 0.19872 | 0.19872 | 0.23077 | 0.23077 |
| F0.5 | 0.75676 | 0.7439 | 0.82915 | 0.8377 | 0.82857 | 0.7907 | 0.81492 | 0.78947 | 0.78571 | 0.75581 |
| F1 | 0.74172 | 0.75776 | 0.83544 | 0.83117 | 0.79452 | 0.81928 | 0.79195 | 0.80982 | 0.75342 | 0.78313 |
| F2 | 0.72727 | 0.77215 | 0.84184 | 0.82474 | 0.76316 | 0.85 | 0.77023 | 0.83123 | 0.72368 | 0.8125 |
| FDR | 0.23288 | 0.26506 | 0.175 | 0.15789 | 0.14706 | 0.22727 | 0.16901 | 0.22353 | 0.19118 | 0.26136 |
| FN | 22 | 17 | 12 | 14 | 20 | 10 | 19 | 12 | 23 | 13 |

**Table 6** continued

| Algo | Bagging | | RandomForest | | DecisionTree | | AdaBoost | | GradientBoost | |
|---|---|---|---|---|---|---|---|---|---|---|
| Class | 0 | 1 | 0 | 1 | 0 | 1 | 0 | 1 | 0 | 1 |
| FNR | 0.28205 | 0.21795 | 0.15385 | 0.17949 | 0.25641 | 0.12821 | 0.24359 | 0.15385 | 0.29487 | 0.16667 |
| FOR | 0.26506 | 0.23288 | 0.15789 | 0.175 | 0.22727 | 0.14706 | 0.22353 | 0.16901 | 0.26136 | 0.19118 |
| FP | 17 | 22 | 14 | 12 | 10 | 20 | 12 | 19 | 13 | 23 |
| FPR | 0.21795 | 0.28205 | 0.17949 | 0.15385 | 0.12821 | 0.25641 | 0.15385 | 0.24359 | 0.16667 | 0.29487 |
| G | 0.74213 | 0.75813 | 0.83551 | 0.83124 | 0.79639 | 0.82077 | 0.79282 | 0.81056 | 0.7552 | 0.78456 |
| GI | 0.5 | 0.5 | 0.66667 | 0.66667 | 0.61538 | 0.61538 | 0.60256 | 0.60256 | 0.53846 | 0.53846 |
| GM | 0.74931 | 0.74931 | 0.83323 | 0.83323 | 0.80514 | 0.80514 | 0.80002 | 0.80002 | 0.76656 | 0.76656 |
| IBA | 0.52548 | 0.59746 | 0.71208 | 0.67648 | 0.56515 | 0.73137 | 0.5826 | 0.69748 | 0.51227 | 0.66294 |
| ICSI | 0.48507 | 0.51699 | 0.67115 | 0.66262 | 0.59653 | 0.64452 | 0.5874 | 0.62262 | 0.51395 | 0.57197 |
| IS | 0.61753 | 0.5557 | 0.72247 | 0.75207 | 0.77052 | 0.62803 | 0.7329 | 0.635 | 0.6939 | 0.56294 |
| J | 0.58947 | 0.61 | 0.71739 | 0.71111 | 0.65909 | 0.69388 | 0.65556 | 0.68041 | 0.6044 | 0.64356 |
| LS | 1.53425 | 1.46988 | 1.65 | 1.68421 | 1.70588 | 1.54545 | 1.66197 | 1.55294 | 1.61765 | 1.47727 |
| MCC | 0.50103 | 0.50103 | 0.66689 | 0.66689 | 0.62051 | 0.62051 | 0.60501 | 0.60501 | 0.54294 | 0.54294 |
| MCCI | Moderate | Moderate | Moderate | Moderate | Moderate | Moderate | Moderate | Moderate | Moderate | Moderate |
| MCEN | 0.93295 | 0.91516 | 0.79663 | 0.80517 | 0.84233 | 0.80391 | 0.8613 | 0.83368 | 0.90256 | 0.86681 |
| MK | 0.50206 | 0.50206 | 0.66711 | 0.66711 | 0.62567 | 0.62567 | 0.60746 | 0.60746 | 0.54746 | 0.54746 |
| N | 78 | 78 | 78 | 78 | 78 | 78 | 78 | 78 | 78 | 78 |
| NLR | 0.36066 | 0.30357 | 0.1875 | 0.21212 | 0.29412 | 0.17241 | 0.28788 | 0.20339 | 0.35385 | 0.23636 |
| NLRI | Poor | Poor | Fair | Poor | Poor | Fair | Poor | Poor | Poor | Poor |
| NPV | 0.73494 | 0.76712 | 0.84211 | 0.825 | 0.77273 | 0.85294 | 0.77647 | 0.83099 | 0.73864 | 0.80882 |

**Table 6** continued

| Algo | Bagging | | RandomForest | | DecisionTree | | AdaBoost | | GradientBoost | |
|---|---|---|---|---|---|---|---|---|---|---|
| Class | 0 | 1 | 0 | 1 | 0 | 1 | 0 | 1 | 0 | 1 |
| OC | 0.76712 | 0.78205 | 0.84615 | 0.84211 | 0.85294 | 0.87179 | 0.83099 | 0.84615 | 0.80882 | 0.83333 |
| OOC | 0.74213 | 0.75813 | 0.83551 | 0.83124 | 0.79639 | 0.82077 | 0.79282 | 0.81056 | 0.7552 | 0.78456 |
| OP | 0.70726 | 0.70726 | 0.81795 | 0.81795 | 0.72833 | 0.72833 | 0.74528 | 0.74528 | 0.6859 | 0.6859 |
| P | 78 | 78 | 78 | 78 | 78 | 78 | 78 | 78 | 78 | 78 |
| PLR | 3.29412 | 2.77273 | 4.71429 | 5.33333 | 5.8 | 3.4 | 4.91667 | 3.47368 | 4.23077 | 2.82609 |
| PLRI | Poor | Poor | Poor | Fair | Fair | Poor | Poor | Poor | Poor | Poor |
| POP | 156 | 156 | 156 | 156 | 156 | 156 | 156 | 156 | 156 | 156 |
| PPV | 0.76712 | 0.73494 | 0.825 | 0.84211 | 0.85294 | 0.77273 | 0.83099 | 0.77647 | 0.80882 | 0.73864 |
| PRE | 0.5 | 0.5 | 0.5 | 0.5 | 0.5 | 0.5 | 0.5 | 0.5 | 0.5 | 0.5 |
| Q | 0.80264 | 0.80264 | 0.9235 | 0.9235 | 0.90347 | 0.90347 | 0.88937 | 0.88937 | 0.84564 | 0.84564 |
| QI | Strong | Strong | Strong | Strong | Strong | Strong | Strong | Strong | Strong | Strong |
| RACC | 0.23397 | 0.26603 | 0.25641 | 0.24359 | 0.21795 | 0.28205 | 0.22756 | 0.27244 | 0.21795 | 0.28205 |
| RACCU | 0.23423 | 0.26628 | 0.25645 | 0.24363 | 0.21898 | 0.28308 | 0.22807 | 0.27294 | 0.21898 | 0.28308 |
| TN | 61 | 56 | 64 | 66 | 68 | 58 | 66 | 59 | 65 | 55 |
| TNR | 0.78205 | 0.71795 | 0.82051 | 0.84615 | 0.87179 | 0.74359 | 0.84615 | 0.75641 | 0.83333 | 0.70513 |
| TON | 83 | 73 | 76 | 80 | 88 | 68 | 85 | 71 | 88 | 68 |
| TOP | 73 | 83 | 80 | 76 | 68 | 88 | 71 | 85 | 68 | 88 |
| TP | 56 | 61 | 66 | 64 | 58 | 68 | 59 | 66 | 55 | 65 |
| TPR | 0.71795 | 0.78205 | 0.84615 | 0.82051 | 0.74359 | 0.87179 | 0.75641 | 0.84615 | 0.70513 | 0.83333 |
| Y | 0.5 | 0.5 | 0.66667 | 0.66667 | 0.61538 | 0.61538 | 0.60256 | 0.60256 | 0.53846 | 0.53846 |
| dInd | 0.35645 | 0.35645 | 0.2364 | 0.2364 | 0.28668 | 0.28668 | 0.28811 | 0.28811 | 0.33871 | 0.33871 |
| sInd | 0.74795 | 0.74795 | 0.83284 | 0.83284 | 0.79729 | 0.79729 | 0.79628 | 0.79628 | 0.76049 | 0.76049 |

impact on future smart contract platform capabilities.These patterns provide baseline data in cryptocurrency domain for academicians and smart contract developers. It showcases the potential of NLP and ML techniques in fortifying the security of blockchain ecosystems. By automating the detection of vulnerabilities in smart contracts, we contribute to the ongoing efforts to ensure the integrity and trustworthiness of decentralized applications. The application of these technologies not only enhances the safety of smart contract development but also promotes the wider adoption and acceptance of blockchain technology in various sectors. As blockchain continues to evolve, so too will our methods for securing it, making our methodology an important step towards a more secure and resilient digital future.

**Data Availability** Data sharing not applicable to this article as no datasets were generated or analyzed during the current study.

## Declarations

**Conflict of Interest** The authors declare that they have no conflict of interest.

## References


1. Dinh TN, Thai MT (2018) AI and blockchain: a disruptive integration. Computer 51(9):48–53
2. Parthasarathy S, Harikrishnan A, Narayanan G, Singh K et al (2021) Secure distributed medical record storage using blockchain and emergency sharing using multi-party computation. In: 2021 11th IFIP international conference on new technologies, mobility and security (NTMS), pp 1–5
3. Zhang L, Xie Y, Zheng Y, Xue W, Zheng X, Xu X (2020) The challenges and countermeasures of blockchain in finance and economics. Syst Res Behav Sci 37(4):691–698
4. Treleaven P, Brown RG, Yang D (2017) Blockchain technology in finance. Computer 50(9):14–17
5. Kar AK, Navin L (2021) Diffusion of blockchain in insurance industry: an analysis through the review of academic and trade literature. Telemat Informat 58:101532
6. Olnes S, Ubacht J, Janssen M (2017) Blockchain in government: benefits and implications of distributed ledger technology for information sharing 355–364
7. Thakur V, Doja M, Dwivedi YK, Ahmad T, Khadanga G (2020) Land records on blockchain for implementation of land titling in India. Int J Inf Manag 52:101940
8. Li C Palanisamy B (2019) Incentivized blockchain-based social media platforms: a case study of steemit. In: Proceedings of the 10th ACM conference on web science, pp 145–154
9. Jing TW Murugesan RK (2018) A theoretical framework to build trust and prevent fake news in social media using blockchain. In: International conference of reliable information and communication technology, pp 955–962
10. Longo F, Nicoletti L, Padovano A, d'Atri G, Forte M (2019) Blockchain-enabled supply chain: an experimental study. Comput Ind Eng 136:57–69
11. Kannan G, Pattnaik M, Karthikeyan G, Augustine BEP, Lohith JJ (2022) Managing the supply chain for the crops directed from agricultural fields using blockchains. In: International conference on electronics and renewable systems (ICEARS)
12. Alammary A, Alhazmi S, Almasri M, Gillani S (2019) Blockchain-based applications in education: a systematic review. Appl Sci 9(12):2400
13. Antonopoulos AM (2017) Mastering Bitcoin: Programming the open blockchain. Reilly Media, Inc
14. Thombs M, Tillman AA (2018) Designing 21st century curriculum for bitcoin and blockchain studies. Int J Global Business 11(1):67–80
15. Swan M (2015) Blockchain: blueprint for a new economy. Reilly Media, Inc
16. Nofer M, Gomber P, Hinz O, Schiereck D (2017) Blockchain. Business Inf. Syst Eng 59(3):183–187
17. Bogner A, Chanson M, Meeuw A (2016) A decentralised sharing app running a smart contract on the ethereum blockchain. In: Proceedings of the 6th international conference on the Internet of Things, pp 177–178
18. Hu T, Liu X, Chen T, Zhang X, Huang X, Niu W, Lu J, Zhou K, Liu Y (2021) Transaction-based classification and detection approach for ethereum smart contract. Inf Process Manag 58(2):102462

19. Szabo N (1994) Smart contracts
20. Nakamoto S (2008) Bitcoin A (2A peer-to-peer electronic cash system). Bitcoin. https://bitcoin.org/bitcoin.pdf
21. Alkhalifah A, Ng A, Kayes A, Chowdhury J, Alazab M, Watters PA (2020) A taxonomy of blockchain threats and vulnerabilities. In: Blockchain for cybersecurity and privacy, CRC Press, pp 3–28
22. Chen H, Pendleton M, Njilla L, Xu S (2020) A survey on ethereum systems security: vulnerabilities, attacks, and defenses. ACM Comput Surveys (CSUR) 53(3):1–43
23. Dingman W, Cohen A, Ferrara N, Lynch A, Jasinski P, Black PE, Deng L (2019) Defects and vulnerabilities in smart contracts, a classification using the NIST bugs framework. Int J Netw Distrib Comput 7(3):121–132
24. Mikolov T, Le QV, Sutskever I. (2013) Exploiting similarities among languages for machine translation. arXiv:1309.4168
25. Manning CD, Raghavan P, Schäutze H (2008) Scoring, term weighting, and the vector space model. Cambridge University Press, pp 100–123
26. Wu HC, Luk RWP, Wong KF, Kwok KL (2008) Interpreting tf-idf term weights as making relevance decisions. ACM Trans Inf Syst (TOIS) 26(3):1–37
27. Luu L, Chu D-H, Olickel H, Saxena P, Hobor A (2016) Making smart contracts smarter. In: ACM SIGSAC conference on computer and communications security, pp 254–269
28. Chen T, Li Z, Zhang Y, Luo X, Wang T, Hu T, X X, Wang D, Huang J, Zhang X (2019) A large-scale empirical study on control flow identification of smart contracts. In: 2019 ACM/IEEE international symposium on empirical software engineering and measurement (ESEM), pp 1–11 Paik JH (2013) A novel tf-idf weighting scheme for effective ranking. In: Proceedings of the 36th international ACM SIGIR conference on research and development in information retrieval, pp 343–352
29. Revere R, Antunes JP (Oct 2018) Solgraph
30. Kushwaha SS, Joshi S, Singh D, Kaur M, H.-N. Lee (2022) Systematic review of security vulnerabilities in ethereum blockchain smart contract. IEEE Access
31. Osterland T, Rose T (2020) Model checking smart contracts for ethereum. Pervas Mobile Comput 63:101129
32. Mueller B Introducing mythril: a framework for bug hunting on the Ethereum blockchain. https://medium.com/hackernoon/introducing-mythril-a-framework-for-bug-hunting-on-the-ethereumblockchain
33. Yu F, Alkhalaf M, Bultan T (2009) Generating vulnerability signatures for string manipulating programs using automata-based forward and backward symbolic analyses. In: 2009 IEEE/ACM international conference on automated software engineering, pp 605–609
34. Raychev V, Vechev M, Yahav E (2014) Code completion with sta istical language models. In: Proceedings of the 35th ACM SIGPLAN conference on programming language design and implementation, pp 419–428
35. Cummins C, Petoumenos P, Murray A, Leather H (2018) Compiler fuzzing through deep learning. In: Proceedings of the 27th ACM SIGSOFT international symposium on software testing and analysis, pp 95–105
36. Tufano M, Watson C, Bavota G, Di Penta M, White M, Poshyvanyk D (2018) An empirical investigation into learning bugfixing patches in the wild via neural machine translation. In: Proceedings of the 33rd ACM/IEEE international conference on automated software engineering, pp 832–837
37. He N, Zhang R, Wang H, Wu L, Luo X, Guo Y, Yu T, Jiang X (2021) Eosafe: security analysis of EOSIO smart contracts. In: 30th USENIX security symposium USENIX security 21
38. Frank J, Aschermann C, Holz T (2020) ETHBMC: a bounded model checker for smart contracts. In: 29th USENIX security symposium USENIX security 20, pp 2757–2774
39. Wang Z, Jin H, Dai W, Choo K-KR, Zou D (2021) Ethereum smart contract security research: survey and future research opportunities. Front Comput Sci 15(2):1–18
40. Mense A, Flatscher M (2018) Security vulnerabilities in ethereum smart contracts. In: Proceedings of the 20th international conference on information integration and web-based applications and services, pp 375–380
41. Nakamura Y, Zhang Y, Sasabe M, Kasahara S (2019) Capabilitybased access control for the Internet of Things: an ethereum blockchain-based scheme. In: 2019 IEEE global communications conference (GLOBECOM), pp 1–6
42. Gupta BC (2019) Analysis of ethereum smart contracts–a security perspective. Indian Inst Technol Kanpur 1–59
43. Tikhomirov S, Voskresenskaya E, Ivanitskiy I, Takhaviev R, Marchenko E, Alexandrov Y (2018) Smartcheck: static analysis of ethereum smart contracts. In: Proceedings of the 1st international workshop on emerging trends in software engineering for blockchain, pp 9–16

44. Chen H, Pendleton M, Njilla L, Xu S (2019) A survey on ethereum systems security: vulnerabilities, attacks and defenses. arXiv:1908.04507
45. Greene R, Johnstone MN (2018) An investigation into a denial of service attack on an ethereum network
46. Su L, Shen X, Du X, Liao X, Wang X, Xing L, Liu B (2021) Evil under the sun: understanding and discovering attacks on ethereum decentralized applications. In: 30th USENIXg security symposium (USENIXg Security 21)
47. Bäunz B, Agrawal S, Zamani M, Boneh D (2020) Zether: towards privacy in a smart contract world. In: International conference on financial cryptography and data security, Springer, pp 423–443
48. Grishchenko I, Maffei M, Schneidewind C (2018) A semantic framework for the security analysis of ethereum smart contracts. In: International conference on principles of security and trust, Springer, pp 243–269
49. Estevam G, Palma LM, Silva LR, Martina JE, Vigil M (2021) Accurate and decentralized timestamping using smart contracts on the ethereum blockchain. Inf Process Manag 58(3):102471
50. Zhang P, Xiao F, Luo X (2020) A framework and dataset for bugs in ethereum smart contracts. In: 2020 IEEE international conference on software maintenance and evolution (ICSME), pp 139–150
51. Durieux T, Ferreira JF, Abreu R, Cruz P (2020) Empirical review of automated analysis tools on 47,587 ethereum smart contracts. In: Proceedings of the ACM/IEEE 42nd international conference on software engineering, pp 530–541
52. Raju RS, Gurung S, Rai P (2022) An overview of 51 attack over bitcoin network. In: Contemporary issues in communication, cloud and big data analytics, pp 39–55
53. Sayeed S, Marco-Gisbert H (2019) Assessing blockchain consensus and security mechanisms against the 51 attack. Appl Sci 9(9):1788
54. Oksiiuk O, Dmyrieva I (2020) Security and privacy issues of blockchain technology. In: 2020 IEEE 15th international conference on advanced trends in radioelectronics, telecommunications and computer engineering (TCSET), pp 1–5
55. Vanhoef M, Piessens F (2017) Key reinstallation attacks: forcing nonce reuse in wpa2. In: Proceedings of the 2017 ACM SIGSAC conference on computer and communications security, pp 1313–1328
56. Sastry JB, Banik BG (2022) Recent trends in blockchain technology. In: Computational intelligence in pattern recognition, Springer, pp 129–142
57. Jonathan K, Sari AK (2019) Security issues and vulnerabilities on a blockchain system: a review. In: 2019 international seminar on research of information technology and intelligent systems (ISRITI), pp 228–232
58. Yahaya AS, Javaid N, Javed MU, Almogren A, Radwan A (2022) Blockchain based secure energy trading with mutual verifiable fairness in a smart community. IEEE Trans Ind Informat
59. Iqbal M, Matulevičius R (2021) Exploring sybil and doublespending risks in blockchain systems. IEEE Access 9:76153–76177
60. Vats VK, Katarya R (2022) Analyzing the attacks on blockchain technologies. In: Machine learning, advances in computing, renewable energy and communication, Springer, pp 379–390
61. Six N, Herbaut N, Salinesi C (2022) Blockchain software patterns for the design of decentralized applications: a systematic literature review. Blockchain: Res Appl 100061
62. Six N, Herbaut N, Salinesi C (2022) Blockchain software patterns for the design of decentralized applications: A systematic literature review. Blockchain: Res Appl 100061
63. Islam MR, Rahman MM, Mahmud M, Rahman MA, Mohamad MHS et al (2021) A review on blockchain security issues and challenges. In: 2021 IEEE 12th control and system graduate research colloquium (ICSGRC), pp 227–232
64. Chiu J, Koeppl TV (2019) Blockchain-based settlement for asset trading. Rev Financ Stud 32(5):1716–1753
65. Al-Jaroodi J, Mohamed N (2019) Blockchain in industries: a survey. IEEE Access 7:36500–36515
66. Singh A, Parizi RM, Zhang Q, Choo K-KR, Dehghantanha A (2020) Blockchain smart contracts formalization: approaches and challenges to address vulnerabilities. Comput Security 88:101654
67. Mikolov T, QV L, Sutskever I (2013) Exploiting similarities among languages for machine translation. arXiv:1309.4168
68. Manning CD, Raghavan P, Schütze H (2008) Scoring, term weighting, and the vector space model. Cambridge University Press, pp 100–123
69. Wu HC, Luk RWP, Wong KF, Kwok KL (2008) Interpreting tf-idf term weights as making relevance decisions. ACM Trans Inf Syst (TOIS) 26(3):1–37
70. Paik JH (2013) A novel tf-idf weighting scheme for effective ranking. In: Proceedings of the 36th international ACM SIGIR conference on Research and development in information retrieval, pp 343–352

71. Ashizawa N, Yanai N, Cruz JP, Okamura S (2021) Eth2vec: learning contract-wide code representations for vulnerability detection on ethereum smart contracts. In Proceedings of the 3rd ACM international symposium on blockchain and secure critical infrastructure, pp 47–59
72. Jeon S, Lee G, Kim H, Woo SS (2021) Smartcondetect: highly accurate smart contract code vulnerability detection mechanism using bert
73. Lutz O, Chen H, Fereidooni H, Sendner C, Dmitrienko A, Sadeghi AR, Koushanfar F (2021) Escort: ethereum smart contracts vulnerability detection using deep neural network and transfer learning. arXiv:2103.12607
74. Ruan X, Feng J, Xu C, Wang J (2020) Observer-based dynamic event-triggered strategies for leader-following consensus of multi-agent systems with disturbances. IEEE Trans Netw Sci Eng 7(4):3148–3158
75. Kalra S, Goel S, Dhawan M, Sharma S (2018) Zeus: analyzing safety of smart contracts. In: Ndss, pp 1–12
76. Bartoletti M, Pompianu L (2017) An empirical analysis of smart contracts: platforms, applications, and design patterns. In: International conference on financial cryptography and data security, Springer, pp 494–509
77. Mikolov T, QV L, Sutskever I (2013) Exploiting similarities among languages for machine translation. arXiv:1309.4168
78. Wu HC, Luk RWP, Wong KF, Kwok KL (2008) Interpreting tf-idf term weights as making relevance decisions. ACM Trans Inf Syst (TOIS) 26(3):1–37
79. ] Paik JH (2013) A novel tf-idf weighting scheme for effective ranking. In: Proceedings of the 36th international ACM SIGIR conference on Research and development in information retrieval, pp 343–352
80. Breiman L (1996) Bagging predictors. Mach Learn 24(2):123–140
81. Lohith JJ, Anusree Manoj K, Guru Nanma P et al (2023) TP-detect: trigram-pixel based vulnerability detection for ethereum smart contracts. Multimedia Tools Appl. https://doi.org/10.1007/s11042-023-15042-4

**Publisher's Note** Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.